Visvesvaraya Technological University (VTU), Belgaum- 590018



A Project Report on

"Intrusion Detection System using Hybrid CNN-LSTM with Explainable AI (SHAP) for Indian Network Databases"

Thesis submitted in partial fulfillment of the curriculum prescribed for the award of the degree of Bachelor of Engineering in

Artificial Intelligence and Machine Learning

Submitted by

1JT23AI030	Maanya B S
1JT23AI042	Ruthwik Sai Ganesh
1JT23AI060	Varshini D N

Under the Guidance of

Dr. Madhu B R
Professor and Head
Department of Artificial Intelligence and Machine Learning



Jyothy Institute of Technology

Department of Artificial Intelligence and Machine Learning

Department of Artificial Intelligence and Machine Learning

CERTIFICATE

This is to certify that the project work titled "Intrusion Detection System using Hybrid CNN-LSTM with Explainable AI (SHAP) for Indian Network Databases" is carried out by Maanya B S (1JT23AI030), Ruthwik Sai Ganesh (1JT23AI042), Varshini D N (1JT23AI060), bonafide students of Bachelor of Engineering at the Jyothy Institute of Technology, Bangalore, in partial fulfillment for the award of the degree of Bachelor of Engineering in Artificial Intelligence and Machine Learning of Visvesvaraya Technological University, Belagavi, during the academic year 2023-27.

Dr. Madhu B R
Professor and Head (Guide)
Dept. of AIML,
JIT, VTU,
Bangalore
Date:

Dr. Madhu B R
Professor and Head
Dept. of AIML,
JIT, VTU,
Bangalore
Date:

Name of the Examiner
1.

Signature of the Examiner

2.

DECLARATION

We, Maanya B S (1JT23AI030), Ruthwik Sai Ganesh (1JT23AI042), Varshini D N (1JT23AI060), are students of the eighth semester B.E. in Artificial Intelligence and Machine Learning at Jyothy Institute of Technology, VTU, and hereby declare that the project titled "Intrusion Detection System using Hybrid CNN-LSTM with Explainable AI (SHAP) for Indian Network Databases" has been carried out by us and submitted in partial fulfillment for the award of the degree of Bachelor of Engineering in Artificial Intelligence and Machine Learning during the academic year 2023-27. Further, the matter presented in the project has not been submitted previously by anybody for the award of any degree or any diploma to any other University, to the best of our knowledge and faith.

ACKNOWLEDGEMENT

It is a great pleasure for us to acknowledge the assistance and support of a large number of individuals who have been responsible for the successful completion of this project work.

First, we take this opportunity to express our sincere gratitude to Jyothy Institute of Technology, VTU for providing us with a great opportunity to pursue our

Signature

Name: Maanya B S
USN: 1JT23AI030
Name: Ruthwik Sai Ganesh
USN: 1JT23AI042
Name: Varshini D N
USN: 1JT23AI060

Place : Bangalore
Date :
Bachelor's Degree in this institution.

In particular we would like to thank Dr. K. Gopalakrishna, Principal, Jyothy Institute of Technology, VTU, for the constant encouragement and expert advice.

It is a matter of immense pleasure to express our sincere thanks to ***Dr. Madhu B R, Professor and Head of the department****,* ***Artificial Intelligence and Machine Learning,*** *for providing right academic guidance that made our task possible.*

We would like to thank our guide and Project Coordinator Dr. Madhu B R, Professor and Head, Department of Artificial Intelligence and Machine Learning, for sparing valuable time to extend help in every step of our project work, and all the staff members of the Department of AIML for their support.

We thank the University of New Brunswick and Manipur University for the CICIDS2017 and MU-IoT datasets, and the owner of the network used for the live test for permission to use it.

We are also grateful to our family and friends who provided us with every requirement throughout the course.

We would like to thank one and all who directly or indirectly helped us in completing the Project work successfully.

Signature of Students

ABSTRACT

Intrusion detection systems built on deep learning are usually reported as accuracy figures on benchmark datasets, and are rarely shown acting on live traffic. This project builds and evaluates a complete intrusion detection and prevention system around a hybrid CNN-LSTM model that classifies windows of consecutive network flows, explains its decisions with SHAP (SHapley Additive exPlanations), and blocks attacking devices through the host firewall in real time. The model is evaluated on the CICIDS2017 benchmark with a chronological, leakage-controlled split, and on MU-IoT, real Internet-of-Things traffic recorded at Manipur University, India, with four whole recordings held out. A random forest trained and tested on identical examples is used as a fair baseline. The forest is better on CICIDS2017 (macro F1 0.780 against 0.586), whereas the CNN-LSTM is clearly better on MU-IoT (held-out macro F1 0.676 against 0.404), where single flow records are ambiguous and sequences resolve them. SHAP is used in the dashboard to explain every prediction and, offline, to separate three causes of the model's errors on unseen IoT recordings.

A per-class prevention policy derives each class's confidence threshold from its measured F1 score and locks unreliable classes to human review; on the CICIDS2017 test set it acts automatically on 96.6% of attack windows and on 0.33% of benign windows. A packet-to-flow extractor reproduces the CICFlowMeter-V3 features on 99.8% of flows from the original CICIDS2017 packet capture, so live traffic and Wireshark captures are analysed with the features the model was trained on. In a live run on a shared Wi-Fi network, a real port scan was detected about two seconds after its first probes and the scanning device was blocked automatically, while another user's falsely flagged device was refused by the safety checks. The limitations of a 2017-trained model on present-day traffic are measured and reported.

Keywords: intrusion detection, intrusion prevention, CNN-LSTM, deep learning, explainable AI, SHAP, CICIDS2017, MU-IoT, live packet capture.

TABLE OF CONTENTS

Chapter 1 Introduction 1
1.1 Introduction 1
1.1.1 Network Intrusions 1
1.1.2 Intrusion Detection Systems 3
1.1.3 Intrusion Prevention 3
1.1.4 Machine Learning and Deep Learning for Intrusion Detection 4
1.1.5 Explainable Artificial Intelligence 4
1.2 Problem Statement 5
1.3 Motivation 5
1.4 Objectives 5
1.5 Scope of the Project 6
1.6 Organisation of the Report 7
Chapter 2 Literature Survey 8
2.1 Literature Survey 8
2.1.1 Foundations of Intrusion Detection 8
2.1.2 Benchmark Datasets and Flow Features 9
2.1.3 IoT Intrusion Datasets 10
2.1.4 Deep Learning for Intrusion Detection 11
2.1.5 Hybrid CNN-LSTM Models 11
2.1.6 Explainable AI for Intrusion Detection 12
2.1.7 Classical Baselines 12
2.2 Summary of the Literature Survey 13
2.3 Research Gap 15
Chapter 3 Objective and Methodology 16
3.1 Objective 16
3.2 Methodology 17
3.2.1 Overview 17
3.2.2 Datasets 17
3.2.3 Preprocessing and Feature Selection 19
3.2.4 Evaluation Design 20
3.2.5 Model Development 21
3.2.6 Explainability Method 22
3.2.7 Prevention Policy Design 23
3.2.8 Live Capture and Packet-to-Flow Extraction 23
3.2.9 Dashboard and Backend 24
3.2.10 Evaluation Metrics 24
Chapter 4 System Design 26
4.1 System Architecture 26
4.2 Use Case Diagram 28
4.3 Data Flow Diagram 29

4.4 Sequence Diagram 31
4.5 Algorithms Used 35
4.5.1 Packet-to-Flow Extraction 35
4.5.2 Window Construction and Scaling 36
4.5.3 Hybrid CNN-LSTM Classifier 36
4.5.4 Random-Forest Baseline 39
4.5.5 SHAP Explanations 39
4.5.6 Prevention Policy 40
4.5.7 Target Selection and Firewall Execution 42
Chapter 5 Hardware and Software Requirement 44
5.1 Hardware Requirement 44
5.2 Software Requirement 45
Chapter 6 Implementation, Testing and Results 46
6.1 Implementation 46
6.1.1 Code Organisation 46
6.1.2 Dashboard 46
6.1.3 Prevention in Operation 48
6.2 Testing 50
6.2.1 Automated Tests 50
6.2.2 Defects Found by Testing 52
6.3 Results 52
6.3.1 Detection on CICIDS2017 52
6.3.2 Detection on MU-IoT 55
6.3.3 Explainability 56
6.3.4 Prevention Policy 58
6.3.5 Validation of the Flow Extractor 58
6.3.6 Real Traffic and Public Captures 59
6.3.7 Live Run on a Real Network 61
6.3.8 Inference Speed 62
6.4 Discussion 62
6.5 Limitations 63
Conclusion and Future Enhancements 65
Conclusion 65
Future Enhancements 66
References 67
Appendix A Backend API Reference 71
Appendix B MU-IoT Features and Dashboard Categories 72
B.1 MU-IoT Features 72
B.2 Dashboard Categories 72

LIST OF FIGURES

Fig. 3.1. Proposed block diagram: offline development and online operation 17
Fig. 4.1. System architecture 27
Fig. 4.2. Use case diagram 29
Fig. 4.3(a). Data flow diagram, level 0 30
Fig. 4.3(b). Data flow diagram, level 1 30
Fig. 4.4(a). Sequence diagram: live detection and automatic blocking 32
Fig. 4.4(b). Sequence diagram: analysis of an uploaded capture 33
Fig. 4.4(c). Sequence diagram: SHAP explanation of a window 34
Fig. 4.4(d). Sequence diagram: revoking a firewall block 35
Fig. 4.5. Architecture of the hybrid CNN-LSTM model with the output shape of each layer 37
Fig. 4.6. Flowchart of the prevention decision for one window 42
Fig. 6.1. Batch analysis of an uploaded flow file in the dashboard 47
Fig. 6.2. Live interface capture in the dashboard during the final demonstration 48
Fig. 6.3. Live run: a port scan detected and blocked, then reported as already blocked 49
Fig. 6.4. Detection log: Bot predictions held for review, PortScan acted on 50
Fig. 6.5. Blocked devices panel with one rule in force and the revoke control 50
Fig. 6.6. Row-normalised confusion matrix of the CNN-LSTM on the CICIDS2017 test set 54
Fig. 6.7. Training and validation curves of the deployed CICIDS2017 model 54
Fig. 6.8. Mean SHAP values of normal traffic flagged as Spyware, real Spyware detections and correctly classified normal traffic 57

LIST OF TABLES

Table 1.1. Traffic classes of the CICIDS2017 dataset 2
Table 1.2. Traffic classes of the MU-IoT dataset used in this project 2
Table 2.1. Summary of the surveyed literature 13
Table 3.1. Cleaning of the CICIDS2017 capture files 18
Table 3.2. The 20 CICIDS2017 features selected by random-forest importance 19
Table 3.3. MU-IoT rows per class and split 21
Table 3.4. Class-weighting schemes compared on the CICIDS2017 test set 22
Table 3.5. CICFlowMeter-V3 behaviour reproduced by the extractor 23
Table 4.1. Components of the system and their implementation 28
Table 4.2. Layers of the CNN-LSTM models 37
Table 4.3. Prevention policy for the CICIDS2017 classes 40
Table 5.1. Hardware used in the project 44
Table 5.2. Minimum hardware requirements for running the system 44
Table 5.3. Software used in the project 45
Table 6.1. Automated tests 51
Table 6.2. CICIDS2017: CNN-LSTM and random forest on identical test examples 52
Table 6.3. CICIDS2017: F1 score per class 52
Table 6.4. MU-IoT: CNN-LSTM and random forest on unseen test windows 55
Table 6.5. MU-IoT: accuracy on each held-out recording 55
Table 6.6. Measured comparison of SHAP explainers on the deployed model 56
Table 6.7. Causes of the main MU-IoT errors found with SHAP 56
Table 6.8. Effect of the prevention policy on all test windows 58
Table 6.9. Extractor against CICFlowMeter-V3 on the original CICIDS2017 capture 59
Table 6.10. Predicted attack ratio on public captures 59
Table 6.11. Results on own packet captures 60
Table 6.12. Live run on a shared Wi-Fi network 61
Table 6.13. Inference speed on the laptop CPU (AMD Ryzen 3 5300U) 62
Table A.1. Endpoints of the backend API 71
Table B.1. The 38 MU-IoT features 72
Table B.2. Mapping of CICIDS2017 classes to dashboard categories 72

NOMENCLATURE

The abbreviations and symbols used in this report are listed below. Symbols are also defined where they first appear.

Abbreviation	Meaning
API	Application Programming Interface
CICIDS2017	Canadian Institute for Cybersecurity Intrusion Detection Evaluation Dataset 2017
CNN	Convolutional Neural Network
CERT-In	Indian Computer Emergency Response Team
CSV	Comma-Separated Values
DDoS / DoS	Distributed Denial of Service / Denial of Service
F1	Harmonic mean of precision and recall
FIN / RST / SYN	TCP flags: finish, reset, synchronise
IAT	Inter-Arrival Time
IDS / IPS	Intrusion Detection System / Intrusion Prevention System
IoT / IIoT	Internet of Things / Industrial Internet of Things
IPv4 / IPv6	Internet Protocol version 4 / version 6
LAN	Local Area Network
LIME	Local Interpretable Model-agnostic Explanations
LSTM	Long Short-Term Memory
MiTM	Man-in-the-Middle
ML / DL	Machine Learning / Deep Learning
MQTT	Message Queuing Telemetry Transport
MU-IoT	Manipur University IoT intrusion dataset
NIDS	Network Intrusion Detection System
Npcap	Windows packet capture driver used by Wireshark
pcap / pcapng	Packet capture file formats
QUIC	UDP-based transport protocol used by modern web traffic
ReLU	Rectified Linear Unit
RF	Random Forest
RFC 5737	Internet standard that reserves three address ranges for documentation (TEST-NET)
SHAP	SHapley Additive exPlanations
SSDP	Simple Service Discovery Protocol
TCP / UDP	Transmission Control Protocol / User Datagram Protocol
XAI	Explainable Artificial Intelligence
XSS	Cross-Site Scripting

Symbol	Meaning
x, z; μ, σ	Raw and standardised feature values; mean and standard deviation
X_t	Window of ten consecutive standardised flows ending at flow t
K, N, N_k	Number of classes, examples, and examples of class k
w_k	Class weight of class k in the training loss
$\hat{y}_k$, c	Predicted probability of class k; confidence (largest probability)
P_k, R_k, $F1_k$	Precision, recall and F1 score of class k
f_t, i_t, o_t	LSTM forget, input and output gates at step t
c_t, h_t	LSTM cell state and hidden state at step t
$\sigma(\cdot)$, $\tanh(\cdot)$	Logistic sigmoid and hyperbolic tangent
φ_i, φ_0	SHAP value of feature i; base value (expected model output)
θ_k	Automatic-action confidence threshold of class k
$\mathcal{R}$	Set of classes locked to human review
L_i, t_i	Length and timestamp of the i-th packet of a flow

Chapter 1

Introduction

1.1 Introduction

Computer networks carry almost every service an organisation depends on, from email and file sharing to industrial control and the growing number of Internet-of-Things (IoT) devices in homes, campuses and hospitals. Every one of these services is reachable over the network, and every one can be probed, overloaded or broken into from it. An attacker usually begins by scanning a network for open services, continues by guessing passwords or exploiting a vulnerable application, and may finish by flooding a server until it stops answering legitimate users. Each of these steps leaves traces in the traffic: unusual numbers of connections, unusual packet sizes, unusual timing. An intrusion detection system (IDS) watches the traffic for such traces and raises an alert when it finds them.

This report describes the design, implementation and evaluation of an intrusion detection and prevention system built around a hybrid deep-learning model that combines a convolutional neural network (CNN) with a long short-term memory (LSTM) network. The model reads short sequences of network flows, decides whether they are benign or belong to one of several attack classes, and explains each decision with SHAP (SHapley Additive exPlanations). A prevention layer turns trusted decisions into Windows Firewall rules that block the attacking device, and a web dashboard shows the predictions, the explanations and the blocked devices. The system works on recorded datasets, on packet captures made with Wireshark, and on live traffic captured from the network interface of the protected computer.

1.1.1 Network Intrusions

A network intrusion is any unauthorised activity on a network: an attempt to discover, access, disrupt or misuse systems that the attacker has no right to use. The attacks studied in this project fall into a few families, each with its own traffic pattern: reconnaissance (a port scan produces many short flows from one source, most answered by a reset), denial of service (floods of requests, or many connections held open with very slow requests), brute force (automated password

guessing against FTP, SSH or a web login), web application attacks (cross-site scripting and SQL injection), exploits, botnets and infiltration. The CICIDS2017 dataset [1], the main benchmark used in this project, contains fourteen attack classes from these families (Table 1.1). The MU-IoT dataset [2], recorded on an IoT testbed at Manipur University, India, contains six attack classes against IoT devices and services (Table 1.2).

Table 1.1. Traffic classes of the CICIDS2017 dataset

Class	Family	Description
BENIGN	Normal	Ordinary user activity (web, email, file transfer)
PortScan	Reconnaissance	Scans of the victim's ports with nmap
DDoS	Denial of service	Flood from several attacking machines (LOIC)
DoS Hulk	Denial of service	Flood of unique HTTP requests
DoS GoldenEye	Denial of service	HTTP flood that keeps connections alive
DoS slowloris	Denial of service	Many connections held open with partial requests
DoS Slowhttptest	Denial of service	Slow HTTP requests and slow reading of replies
FTP-Patator	Brute force	Password guessing against an FTP server
SSH-Patator	Brute force	Password guessing against an SSH server
Web Attack - Brute Force	Web attack	Password guessing against a web login form
Web Attack - XSS	Web attack	Cross-site scripting through a vulnerable web application
Web Attack - Sql Injection	Web attack	SQL injection through a vulnerable web application
Heartbleed	Exploit	Memory disclosure through the OpenSSL Heartbleed flaw
Bot	Botnet	Infected hosts contacting the Ares botnet's command server
Infiltration	Infiltration	Internal host compromised and used to scan the network

Table 1.2. Traffic classes of the MU-IoT dataset used in this project

Class	Description
normal	Benign IoT and user traffic on the testbed
DDoS	Flooding attacks against IoT services (for example UDP floods)
Scan	Port and vulnerability scans of IoT devices
Password_Hacking	Dictionary and brute-force attacks on IoT services such as MQTT
Injection	Application-layer injection attacks such as cross-site

Class	Description
	scripting
MiTM	Man-in-the-middle interception of IoT traffic
Spyware	Spyware activity on IoT devices

1.1.2 Intrusion Detection Systems

Intrusion detection systems are commonly divided along two lines [3], [4]. By the data they observe, a host-based IDS inspects the logs and processes of one computer, whereas a network-based IDS (NIDS) inspects the traffic passing a point of the network. By the way they decide, a signature-based IDS matches traffic against known attack patterns and misses attacks it has no signature for, whereas an anomaly-based IDS learns what normal traffic looks like and reports departures from it, at the cost of more false alarms [5].

Most modern NIDS research works on flows rather than on individual packets. A flow is the group of packets exchanged between two endpoints in one connection, identified by the source and destination addresses and ports and the protocol. Tools such as CICFlowMeter [6] summarise every flow with dozens of statistical features: its duration, the number and sizes of packets in each direction, and the times between packets. Flow features are compact, they do not require reading the encrypted payload, and they are the representation in which the CICIDS2017 dataset is published.

1.1.3 Intrusion Prevention

An intrusion prevention system (IPS) goes one step further than detection: it acts on its alerts, for example by dropping the attacker's packets or blocking its address in a firewall. Acting is more useful than alerting, because a scan or a flood is stopped instead of merely recorded, but it is also riskier. A detection model that is wrong 1% of the time on benign traffic produces a steady stream of false alarms; a prevention system with the same error rate cuts legitimate users off the network. Any system that blocks automatically must therefore decide when a prediction is reliable enough to act on, and must keep a human in control of the cases where it is not.

1.1.4 Machine Learning and Deep Learning for Intrusion Detection

Machine-learning classifiers learn the boundary between benign and malicious traffic from labelled examples instead of hand-written signatures. Classical models such as random forests [7] work on one flow at a time and remain strong baselines. Deep-learning models learn their own intermediate representations: convolutional networks [8] detect local patterns, and recurrent networks such as the LSTM [9] model how a sequence develops over time. Studies across several datasets have found deep models competitive with classical ones [10], [11], and hybrid CNN-LSTM architectures are a common choice for sequences of flows [12], because an attack often shows itself not in one flow but in a run of similar flows from the same source.

High benchmark scores have to be read with care. Neighbouring flows in a capture are often near-identical, so a random split of the data places near-copies in both the training and the test set and rewards memorisation; benign traffic dominates most datasets, so overall accuracy hides failures on rare attacks; and a model that does well on traffic recorded in a laboratory may behave differently on a real network [13], [14]. This project treats these pitfalls as part of the problem rather than as footnotes.

1.1.5 Explainable Artificial Intelligence

A deep network does not explain itself. When it flags a device as an attacker, the analyst who has to decide whether to block that device needs to know why. Explainable artificial intelligence (XAI) provides methods that attribute a model's prediction to its inputs. SHAP [15] is the most widely used of them: it assigns each input feature a contribution, based on Shapley values from cooperative game theory, such that the contributions add up exactly to the model's output. In this project SHAP serves two purposes: the dashboard shows the contributions behind every prediction, and an offline analysis uses them to find out why the model makes the mistakes it makes.

1.2 Problem Statement

Deep-learning intrusion detectors are usually evaluated as accuracy figures on benchmark datasets. Such figures do not show whether a detector is better than a simple classical model trained on the same examples, whether it recognises attack recordings it has never seen, why it fails when it fails, or whether it can work on live traffic at all. A detector that is to protect a network must in addition act on its decisions, which raises a question that benchmark scores do not answer: when may a model's prediction be trusted to block a device automatically, and how is the damage limited when it is wrong?

The problem addressed by this project is therefore to build a hybrid CNN-LSTM intrusion detection system that (a) is evaluated honestly against a fair baseline with leakage-controlled data splits, on a benchmark dataset and on Indian IoT traffic; (b) explains its decisions; (c) prevents attacks automatically only where its measured reliability justifies it; and (d) works on live network traffic and on real packet captures, not only on prepared test files.

1.3 Motivation

The first version of this system analysed flow records uploaded as CSV files taken from the CICIDS2017 test set. At the project review, the guide asked for two changes: the input had to be live traffic, and it had to come from real captures made with free tools such as Wireshark rather than from prepared test data. The team added a third requirement of its own: detection alone would not stop an attack, so the system should also prevent it, live.

Meeting these requirements turned out to be the most instructive part of the project: live use requires exactly the training features, quirks included; automatic blocking on a shared network requires safety limits; and present-day traffic exposes the gap between benchmark data and real networks. The title of the project refers to Indian network databases because the second dataset, MU-IoT, was recorded at Manipur University, India, and is used to test whether the approach carries over to IoT traffic.

1.4 Objectives

The objectives of the project, detailed in Section 3.1, are to:

1. Detect fourteen attack classes from windows of consecutive network flows with a hybrid CNN-LSTM model.
2. Evaluate the model with leakage-controlled splits against a random-forest baseline on identical examples, on CICIDS2017 and on the Indian MU-IoT dataset.
3. Explain the model's predictions with SHAP, both per prediction in the dashboard and as an analysis of its errors.
4. Prevent attacks through a per-class policy whose automatic actions are limited by measured reliability and by explicit safety checks.
5. Operate on live traffic and on Wireshark captures, through a packet-to-flow extractor validated against the original dataset.

1.5 Scope of the Project

The project covers the full path from data to action. Within that path its scope is deliberately bounded, and the bounds are stated here so that the results can be read correctly.

- **Data.** Two existing, published datasets are used: CICIDS2017 and MU-IoT. No new labelled dataset was collected. Own traffic was captured with Wireshark and live, but it carries no ground-truth labels and is used to test the system's behaviour, not to measure accuracy.
- **Deployment.** The system runs on the computer it protects (a Windows laptop) and blocks devices through that computer's own firewall. It is not a network appliance placed in front of other machines, and it is not hosted in the cloud.
- **Prevention.** Prevention is deliberately limited (called IPS-lite in this report): rules are created only for documentation addresses reserved by RFC 5737 or for local devices listed explicitly for a demonstration, and only when a kill switch is turned on.
- **Live attacks.** One attack type, a TCP port scan, was run live against the system, on one network, from one device. The other attack classes need victim servers, vulnerable services or several attacking machines on an isolated laboratory network, and were evaluated on the recorded datasets only.

- **Dashboard.** The dashboard serves the CICIDS2017 model, because live traffic is converted into CICIDS2017 features. The MU-IoT model is evaluated offline.

1.6 Organisation of the Report

Chapter 2 surveys the literature on intrusion detection datasets, deep-learning detectors, hybrid CNN-LSTM models and explainable AI, and identifies the gap this project addresses. Chapter 3 states the objectives and describes the methodology: the datasets, preprocessing, evaluation design, model, explainability, prevention policy, live capture and dashboard. Chapter 4 presents the system design: the architecture, use case, data flow and sequence diagrams, and the algorithms with their equations. Chapter 5 lists the hardware and software requirements. Chapter 6 describes the implementation and testing and reports all results, including the live run. The report closes with conclusions and future enhancements, the references and appendices.

Chapter 2

Literature Survey

2.1 Literature Survey

The survey is organised around the questions the project has to answer: what intrusion detection is and what makes it hard (Section 2.1.1), which datasets exist and what is known about their flaws (Sections 2.1.2 and 2.1.3), which deep-learning models have been applied (Sections 2.1.4 and 2.1.5), and how their decisions can be explained (Section 2.1.6). Section 2.2 summarises the surveyed works in a table, and Section 2.3 states the gap this project addresses. Only what each work states about its own method, data and findings is reported here; numerical results of other papers are not quoted, because they were obtained on different data and splits and are not comparable with ours.

2.1.1 Foundations of Intrusion Detection

Surveys of methods. Buczak and Guven [4] surveyed data-mining and machine-learning methods for cyber-security intrusion detection, covering misuse (signature) and anomaly detection, and described for each family of methods how it has been applied, what data it needs and how expensive it is to train and to run. They emphasised that the field lacks good labelled datasets and that a method's practical value depends on its time complexity and on how often it must be retrained. Khraisat et al. [3] gave a taxonomy of intrusion detection systems that separates signature-based from anomaly-based detection, reviewed the machine-learning techniques used for anomaly detection and the public datasets used to evaluate them, and discussed evasion techniques that attackers use against detectors. García-Teodoro et al. [5] reviewed anomaly-based network intrusion detection, grouping techniques into statistical, knowledge-based and machine-learning approaches, and named the open problems that remain relevant: high false-alarm rates, the difficulty of obtaining representative training data, and the lack of realistic evaluation.

Pitfalls of machine learning in security. Sommer and Paxson [13] argued that machine learning has been far less successful in operational network intrusion detection than in other domains, and explained why: detecting attacks is closer to

finding outliers than to classifying well-defined categories, errors are very costly, there is a semantic gap between an alert and what the operator must do with it, network traffic is extremely diverse, and sound evaluation is hard. Arp et al. [14] examined machine-learning work in computer security systematically and identified ten recurring pitfalls, among them sampling bias, label inaccuracy, data snooping (information from the test data influencing training), inappropriate baselines, inappropriate performance measures, and evaluation in the laboratory only. These two works shaped the evaluation design of this project directly: a chronological split against data snooping, a random-forest baseline on identical examples, macro F1 alongside accuracy, and measurement on real traffic.

2.1.2 Benchmark Datasets and Flow Features

KDD Cup 99 and its problems. Tavallaee et al. [16] analysed the KDD Cup 99 dataset, for many years the standard benchmark, and showed that it contains a very large number of redundant records, which bias learning towards frequent records and make evaluation results unreliable. They proposed a reduced dataset, NSL-KDD, as a remedy. The analysis is an early example of a recurring lesson: a benchmark's flaws are inherited by every model evaluated on it.

UNSW-NB15. Moustafa and Slay [17] created UNSW-NB15 with a commercial traffic generator in a cyber-range environment, combining modern normal traffic with nine families of attacks, and described its feature set and its differences from the older KDD datasets.

CICIDS2017. Sharafaldin et al. [1] generated CICIDS2017 at the Canadian Institute for Cybersecurity. Benign background traffic was produced from profiles of human behaviour, and attacks were run over five working days: brute force against FTP and SSH, several denial-of-service tools, Heartbleed, web attacks, infiltration, a botnet, port scans and DDoS. Flows were extracted with CICFlowMeter, which computes more than 80 features per flow, and labelled. The dataset also publishes the original packet captures, which this project uses to validate its own flow extractor.

CICFlowMeter. The flow-feature tool behind CICIDS2017 was introduced by Lashkari et al. [6] in a study that characterised Tor traffic with time-based flow features. Because the CICIDS2017 features are defined by this tool's behaviour, a

model trained on them can only be applied to new traffic if the new traffic is processed in exactly the same way.

Flaws in CICIDS2017. Engelen, Rimmer and Joosen [18] examined CICIDS2017 and CICFlowMeter in detail and documented errors in both flow construction and labelling. One artefact is the "TCP appendix": after a TCP connection is terminated by a FIN packet, the remaining packets of the connection are collected into a separate, new flow. Their study is directly relevant here: the model in this project was trained on flows built this way, so the project's live extractor deliberately reproduces the artefact instead of correcting it (Section 3.2.8).

Dataset surveys. Ring et al. [19] surveyed network-based intrusion detection datasets and compared them along a common set of properties, such as the type of traffic, the recording environment, labelling and the formats in which the data are published. The survey makes the point that no single dataset is representative of all networks, which supports evaluating on more than one dataset, as this project does.

2.1.3 IoT Intrusion Datasets

IoT devices run different protocols (MQTT, CoAP, RTSP), have limited resources and are often poorly secured, so IoT traffic and IoT attacks differ from those in enterprise datasets such as CICIDS2017.

- **N-BaIoT.** Meidan et al. [20] infected commercial IoT devices with the Mirai and BASHLITE botnets and detected the infected devices from their network traffic with deep autoencoders trained per device on benign traffic only.
- **Bot-IoT.** Koroniotis et al. [21] built a realistic testbed for IoT botnet traffic and network forensics, combining normal traffic with denial of service, distributed denial of service, scanning and information-theft attacks.
- **TON_IoT.** Alsaedi et al. [22] collected telemetry data from IoT and industrial IoT services together with operating-system and network data, labelled for both binary and multi-class detection, and evaluated machine-learning models on it.
- **MU-IoT.** Clinton and Hoque [2] recorded a new IoT intrusion dataset on a testbed at Manipur University, India, covering attacks on both the network

layer and the application layer of IoT devices. It is the Indian dataset used in this project; Section 3.2.2 describes how it was prepared.

Deep learning for IoT attacks. Diro and Chilamkurti [23] proposed a distributed deep-learning scheme for attack detection in the Internet of Things, in which detection is performed close to the devices, and reported that the deep model was more effective than shallow models in their experiments.

2.1.4 Deep Learning for Intrusion Detection

Building blocks. Two architectures underlie the model of this project. LeCun et al. [8] showed how convolutional networks trained by gradient descent learn local features directly from raw input, using shared weights and pooling. Hochreiter and Schmidhuber [9] introduced the long short-term memory, a recurrent unit whose gated memory cell keeps information over long sequences and avoids the vanishing gradients of simple recurrent networks.

Deep detectors. Shone et al. [24] proposed a non-symmetric deep autoencoder for unsupervised feature learning, stacked and combined with a random-forest classifier, and evaluated it on the KDD Cup 99 and NSL-KDD benchmarks. Yin et al. [25] applied a recurrent neural network to NSL-KDD for both binary and multi-class detection and compared it with classical classifiers. Vinayakumar et al. [10] evaluated deep neural networks of different depths against classical machine-learning classifiers on several datasets, including KDD Cup 99, NSL-KDD, UNSW-NB15 and CICIDS2017, and proposed a scalable hybrid framework for monitoring network traffic and host events. Ferrag et al. [11] surveyed deep-learning approaches to intrusion detection, described the datasets used in the field, and compared seven deep-learning models experimentally on two recent datasets.

2.1.5 Hybrid CNN-LSTM Models

Halbouni et al. [12] proposed a hybrid CNN-LSTM network for network intrusion detection: convolutional layers extract spatial features from the flow features and LSTM layers model their temporal dependencies, with batch normalisation and dropout layers added to the network. The model was evaluated in binary and multi-class settings on CICIDS2017, UNSW-NB15 and WSN-DS. Their work is the closest to this project's model. The present project uses a

comparable architecture but differs in what it asks of it: the input is a window of ten consecutive flows rather than one flow, the evaluation uses a chronological split and a baseline on identical examples, a second, IoT dataset with held-out recordings tests generalisation, and the model is taken to live traffic.

2.1.6 Explainable AI for Intrusion Detection

SHAP and LIME. Lundberg and Lee [15] unified several explanation methods as additive feature attribution methods and showed that Shapley values are the only attribution that satisfies three desirable properties: local accuracy (the attributions sum to the model's output), missingness and consistency. They proposed efficient estimators, among them Kernel SHAP, which is model-agnostic. Ribeiro et al. [26] introduced LIME, which explains an individual prediction by fitting a simple interpretable model to the classifier's behaviour in the neighbourhood of that input.

XAI applied to intrusion detection. Arreche et al. [27] proposed E-XAI, a framework for evaluating black-box explanation methods such as SHAP and LIME for network intrusion detection on several datasets and models, using criteria such as descriptive accuracy, sparsity, stability, efficiency, robustness and completeness. Their work shows that explanations must themselves be evaluated, which motivated the measurements behind the dashboard's choice of explainer (Section 6.3.3). Gaspar et al. [28] applied LIME and SHAP to a multi-layer perceptron intrusion detector and studied how useful the explanations are for understanding its decisions. Nascita et al. [29] surveyed explainable AI for network traffic classification, traffic prediction and intrusion detection, and Manivannan [30] analysed explainable-AI-driven machine-learning techniques for network intrusion detection systems. These surveys agree that explanations are needed for analysts to trust and act on detector output, and that most works use explanations to justify correct predictions rather than to diagnose wrong ones.

2.1.7 Classical Baselines

The random forest of Breiman [7] trains many decision trees on bootstrap samples of the data, each split choosing from a random subset of features, and predicts by majority vote. Random forests train quickly, handle imbalanced

tabular data well and need little tuning, which makes them a strong baseline for flow-based detection. In this project the same 20 features and the same split are given to a random forest so that the CNN-LSTM is judged against a capable classical model, as Arp et al. [14] recommend.

2.2 Summary of the Literature Survey

Table 2.1 summarises the surveyed works and what each contributes to this project.

Table 2.1. Summary of the surveyed literature

Ref.	Work (year)	Method or contribution	Data or scope	Use in this project
[4]	Buczak and Guven (2016)	Survey of ML and data-mining methods for intrusion detection	Broad IDS literature	Background, taxonomy
[3]	Khraisat et al. (2019)	Survey and taxonomy of IDS, datasets and evasion	Broad IDS literature	Background
[5]	García-Teodoro et al. (2009)	Survey of anomaly-based NIDS	Anomaly detection	Background, open problems
[13]	Sommer and Paxson (2010)	Why ML struggles in operational NIDS	Operational networks	Evaluation on real traffic
[14]	Arp et al. (2022)	Ten pitfalls of ML in security	Security ML papers	Split design, baseline, metrics
[16]	Tavallaee et al. (2009)	Analysis of KDD Cup 99, NSL-KDD	KDD Cup 99	Benchmark flaws
[17]	Moustafa and Slay (2015)	UNSW-NB15 dataset	Cyber-range traffic	Dataset context
[1]	Sharafaldin et al. (2018)	CICIDS2017 dataset	Five-day testbed capture	Main benchmark
[6]	Lashkari et al. (2017)	CICFlowMeter time-based flow features	Tor traffic	Feature definitions
[18]	Engelen et al. (2021)	Errors in CICIDS2017 and CICFlowMeter	CICIDS2017	Extractor reproduces artefacts
[19]	Ring et al. (2019)	Survey of NIDS datasets	34 datasets compared	Use of two datasets
[	Meidan et al. (2018)	Deep autoencoders for	N-BaIoT	IoT context

Ref.	Work (year)	Method or contribution	Data or scope	Use in this project
20]		IoT botnets		
[21]	Koroniotis et al. (2019)	Bot-IoT dataset	IoT testbed	IoT context
[22]	Alsaedi et al. (2020)	TON_IoT telemetry dataset	IoT and IIoT	IoT context
[2]	Clinton and Hoque (2024)	MU-IoT dataset	IoT testbed, Manipur University	Second, Indian dataset
[23]	Diro and Chilamkurti (2018)	Distributed deep learning for IoT attacks	IoT	IoT deep learning
[8]	LeCun et al. (1998)	Convolutional networks	Document recognition	CNN component
[9]	Hochreiter and Schmidhuber (1997)	Long short-term memory	Sequence learning	LSTM component
[24]	Shone et al. (2018)	Non-symmetric deep autoencoder with RF	KDD Cup 99, NSL-KDD	Deep IDS context
[25]	Yin et al. (2017)	Recurrent neural network IDS	NSL-KDD	Sequence models
[10]	Vinayakumar et al. (2019)	Deep networks vs classical ML	Several incl. CICIDS2017	Deep IDS context
[11]	Ferrag et al. (2020)	Survey and comparison of deep IDS	35 datasets surveyed	Deep IDS context
[12]	Halbouni et al. (2022)	Hybrid CNN-LSTM IDS	CICIDS2017, UNSW-NB15, WSN-DS	Closest prior model
[7]	Breiman (2001)	Random forests	General	Baseline model
[15]	Lundberg and Lee (2017)	SHAP, Kernel SHAP	General	Explanations
[26]	Ribeiro et al. (2016)	LIME	General	Alternative explainer
[2	Arreche et al. (2024)	Evaluating XAI for NIDS (E-XAI)	Three NIDS datasets	Explainer evaluation

Ref.	Work (year)	Method or contribution	Data or scope	Use in this project
7]				
[28]	Gaspar et al. (2024)	LIME and SHAP on an MLP IDS	IDS	XAI for IDS
[29]	Nascita et al. (2025)	Survey of XAI for traffic analysis and IDS	Traffic, IDS	XAI context
[30]	Manivannan (2026)	Analysis of XAI-driven ML for NIDS	NIDS	XAI context

2.3 Research Gap

The surveyed literature establishes that machine-learning and deep-learning models can detect intrusions in benchmark datasets, that hybrid CNN-LSTM architectures are a reasonable choice for flow data, and that explanation methods such as SHAP can make their decisions interpretable. It also shows four gaps that this project addresses.

1. **Evaluation.** Most deep-learning studies report accuracy on random splits of one dataset. Few use a chronological split, compare against a classical model on identical examples, or test on whole recordings never seen in training, although the pitfalls this causes are well documented [13], [14].
2. **Explanations of errors.** Explainable AI for intrusion detection mostly explains correct predictions. Using explanations to diagnose why a model fails, and to decide which classes should not be trusted, is rarely done.
3. **From detection to safe prevention.** Few works connect a detector's measured reliability to the decision to act automatically, or describe the safety limits that automatic blocking on a real network requires.
4. **From benchmark to live traffic.** Models trained on CICIDS2017 are rarely run on live traffic, and when they are, the flow features are not checked against the tool that produced the training data, even though the dataset's own flow construction is known to contain artefacts [18].

Chapter 3

Objective and Methodology

3.1 Objective

The aim of the project is a hybrid CNN-LSTM intrusion detection and prevention system that is evaluated honestly, explains its decisions, acts only where it is reliable, and works on live traffic. The specific objectives are:

1. **Detection.** Train a hybrid CNN-LSTM model that classifies windows of consecutive network flows into benign traffic and fourteen attack classes of CICIDS2017, and a second model of the same architecture for the seven classes of the Indian MU-IoT dataset.
2. **Leakage-controlled evaluation.** Split the data so that near-identical neighbouring flows cannot appear on both sides of the split; compare the CNN-LSTM with a random-forest baseline trained and scored on identical examples; report macro F1 and per-class scores alongside accuracy; and test generalisation on whole MU-IoT recordings never used in training.
3. **Explainability.** Explain every prediction in the dashboard with SHAP, choosing and configuring the explainer by measurement, and use SHAP offline to find the causes of the model's main errors.
4. **Prevention.** Map each class to a countermeasure through a policy whose automatic-action thresholds follow each class's measured F1 score, lock unreliable classes to human review, and execute approved actions as Windows Firewall rules behind independent safety checks that make every action reversible.
5. **Live operation.** Convert live traffic and Wireshark captures into the exact features the model was trained on, validate this conversion against the original CICIDS2017 packet capture, and detect and block a real attack on a real network in real time.
6. **Presentation.** Provide a web dashboard that shows predictions, confidence, explanations, prevention decisions and blocked devices, with a control to revoke any block.

3.2 Methodology

3.2.1 Overview

The work was divided among the three team members by area: data preparation and live traffic captures (Maanya B S); model development, evaluation, error analysis, prevention and the live capture engine (Ruthwik Sai Ganesh); and the explainability service, backend and dashboard (Varshini D N). The methodology has an offline part, in which the datasets are prepared and the models are trained and evaluated, and an online part, in which traffic is captured, converted into flows, classified, explained and acted upon (Fig. 3.1).

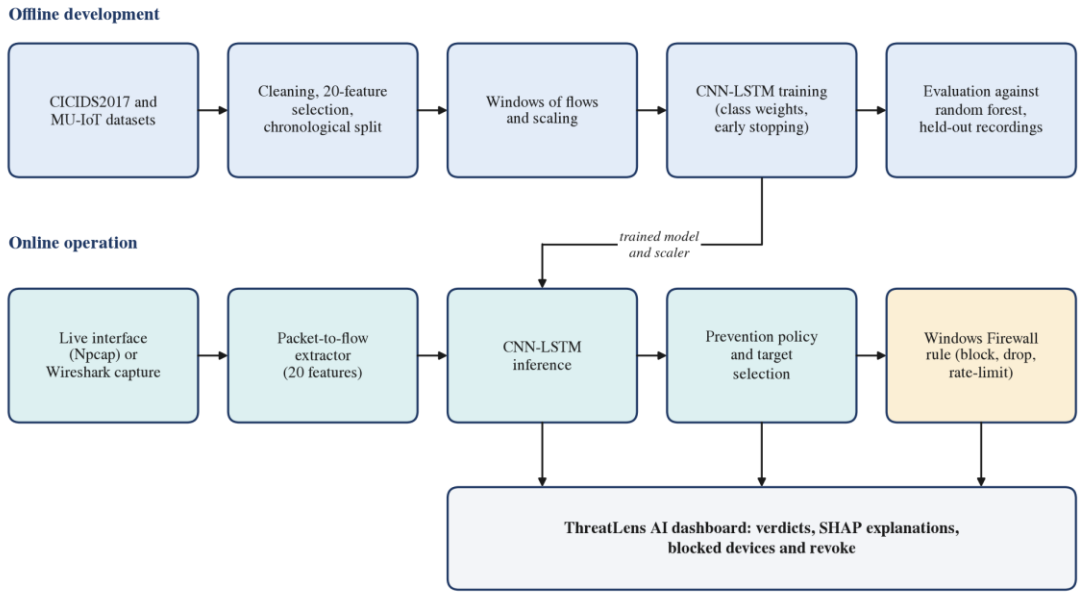


Fig. 3.1. Proposed block diagram: offline development and online operation

The remainder of this section follows the order of Fig. 3.1: datasets and preprocessing (Sections 3.2.2 and 3.2.3), the evaluation design (Section 3.2.4), the model and its training (Section 3.2.5), explainability (Section 3.2.6), the prevention policy (Section 3.2.7), live capture and flow extraction (Section 3.2.8), the dashboard (Section 3.2.9) and the evaluation metrics (Section 3.2.10).

3.2.2 Datasets

CICIDS2017. The dataset [1] is published as eight CSV files, one or two per capture day, with about 79 columns per flow. Each file was loaded, column names were standardised, rows with missing or infinite values and exact duplicate rows were removed, ten columns that were constant across the data

were dropped, the label was encoded as an integer, and the files were combined with a `source_day` column that records which capture file each row came from. Table 3.1 lists the effect of cleaning on each file. The cleaned dataset has 2,655,060 rows and 70 columns.

Table 3.1. Cleaning of the CICIDS2017 capture files

Capture file	Original rows	Missing values	Duplicates removed	Cleaned rows
Monday-WorkingHours	529,918	874	14,278	515,203
Tuesday-WorkingHours	445,909	528	13,859	431,786
Wednesday-WorkingHours	692,703	2,594	66,975	624,431
Thursday-Morning-WebAttacks	170,366	270	3,719	166,512
Thursday-Afternoon-Infiltration	288,602	414	23,225	265,170
Friday-Morning	191,033	244	4,456	186,455
Friday-Afternoon-PortScan	286,467	742	44,734	241,362
Friday-Afternoon-DDoS	225,745	68	1,570	224,141
Total	**2,830,743**	**5,734**	**172,816**	**2,655,060**

The missing-value column counts values, not rows: every row removed for missing data had two missing values, so 2,867 rows were removed for this reason in total (for example, 874 values in 437 rows of the Monday file).

MU-IoT. The MU-IoT dataset [2] was recorded on an IoT testbed at Manipur University. The copy used in this project consists of 40 raw CSV files, about 12.4 GB, with 124 columns. A session-mapping step grouped the 40 files into 32 capture sessions (one recording of one activity), and the session identifier was kept with every row. Cleaning processed the files in chunks and produced 24,171,263 rows with 118 columns: eight completely empty columns and seven constant columns were removed, and the `label` and `type` columns were removed from the features because they encode the target. HTTP and RTSP request methods were encoded as small integers (for example GET as 0 and POST as 1, with -1 for missing), and the source and destination IP addresses were replaced by six numeric indicators each (private, loopback, multicast and link-local flags, and the first and last octets), so that the model cannot learn the testbed's specific addresses.

The published description of the dataset [2] reports a larger record count and 121 flow features; the files used here are the copy available to the team, reduced as

described above and in Section 3.2.3. *[To confirm with the data team: how the 40-file copy relates to the full published dataset.]*

Data preparation also found that two of the raw files (`Botnet_Slowloris.csv` and `Slowloris.csv`) are identical, so they cannot be treated as independent recordings.

3.2.3 Preprocessing and Feature Selection

CICIDS2017 features. Twenty features were selected by random-forest importance. A random forest with 100 trees was trained on a random sample of 100,000 rows of the encoded dataset, the features were ranked by their importance, and the top 20 were kept (Table 3.2). All twenty are statistics of packet lengths, inter-arrival times and the destination port; none of them identifies a particular host.

Table 3.2. The 20 CICIDS2017 features selected by random-forest importance

Rank	Feature	Importance	Rank	Feature	Importance
1	Bwd Packet Length Mean	0.0568	11	Subflow Fwd Bytes	0.0366
2	Packet Length Variance	0.0549	12	Avg Bwd Segment Size	0.0335
3	Packet Length Std	0.0490	13	Fwd IAT Std	0.0304
4	Average Packet Size	0.0454	14	Fwd Packet Length Mean	0.0239
5	Total Length of Bwd Packets	0.0424	15	Subflow Bwd Bytes	0.0217
6	Bwd Packet Length Std	0.0416	16	Flow IAT Max	0.0215
7	Bwd Packet Length Max	0.0410	17	Avg Fwd Segment Size	0.0213
8	Total Length of Fwd Packets	0.0406	18	Fwd Packet Length Max	0.0199
9	Packet Length Mean	0.0390	19	Destination Port	0.0180
10	Max Packet Length	0.0390	20	Subflow Fwd Packets	0.0149

The ranking was computed on a sample of the whole dataset, before the chronological split described below, so a small part of the sample consists of rows that later became test rows. This is a mild selection leak; its effect is probably small, because the ranking used only 100,000 of 2,655,060 rows, but it was not measured, and it is listed among the limitations (Section 6.5).

MU-IoT features. The MU-IoT features were selected from the 48 top-ranked features of the dataset's reference paper [2]. Features with poor coverage or constant values were removed, leaving 40, and two timing features, FPT and LPT (the times of a flow's first and last packet), were then removed because every

recording covers one class and its own period of time, so absolute timestamps would reveal the label. The final set has 38 features (listed in Appendix B).

Scaling. Features are standardised with a `StandardScaler` fitted on the training rows only, so that no information from the validation or test data reaches the model through the scaling parameters:

$$z_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j} \tag{3.1}$$

where x is a raw value of feature j, and μ and σ are that feature's mean and standard deviation over the training rows.

Windows. The CNN-LSTM does not classify one flow but a window of consecutive flows. For CICIDS2017 a window holds the ten most recent flows, each described by its 20 standardised features, and is labelled with the class of its last flow:

$$\mathbf{X}_t = [\,\mathbf{z}_{t-9},\ \mathbf{z}_{t-8},\ \ldots,\ \mathbf{z}_t\,] \in \mathbb{R}^{10\times 20} \tag{3.2}$$

The window slides by one flow, so every flow (after the first nine of a capture day) ends exactly one window. A window never spans two capture days. MU-IoT uses windows of 20 rows with 38 features, which never cross a recording block, a split boundary or a gap in the original row identifiers.

3.2.4 Evaluation Design

The evaluation design matters more than the architecture: a careless split measures memory, not detection [14].

Chronological split for CICIDS2017. CICIDS2017 rows have no reliable timestamp in the cleaned data, but the row order within each capture file preserves the order of capture. The data were therefore split chronologically within each group of (capture day, class): the first 80% of the rows of each group train the model, and the remaining 20% test it. A pure day-level split, holding out whole capture days, was considered and rejected, because each attack appears on only one day and such a split removed eight of the fifteen classes from the test set. The resulting test set has 530,951 windows, 83% of them benign.

Tracks for MU-IoT. Four complete capture sessions were held out from training and validation: an hping3 UDP flood (DDoS), a vulnerability scan (Scan), an

MQTT dictionary attack (Password_Hacking) and an XSS attack (Injection). The remaining sessions were split chronologically within each session into about 70% training, 15% validation and 15% testing. This gives two test tracks (Table 3.3): *within*, later traffic of recordings seen in training, and *held-out*, recordings never seen, which measures generalisation to new attack tools.

Table 3.3. MU-IoT rows per class and split

Class	Train	Validation	Test (within)	Test (held-out)
DDoS	496,750	99,261	149,984	200,000
Injection	165,355	35,433	35,435	132,764
MiTM	18,623	3,991	3,992	0
Password_Hacking	500,000	100,000	149,174	200,000
Scan	500,000	100,000	144,179	200,000
Spyware	95,071	20,373	20,373	0
normal	396,196	84,899	84,900	0
Total	**2,171,995**	**443,957**	**588,037**	**732,764**

MiTM, Spyware and normal traffic have no held-out recording, so their scores come from the within track only. The final MU-IoT feature matrix has 3,936,753 rows and 38 features.

Unseen-only scoring. With timestamps removed, many MU-IoT windows are exact repeats of windows in the training data. Headline MU-IoT scores therefore count only test windows whose exact 20-row sequence never appears in training, so that memorised duplicates cannot inflate them.

Baseline on identical examples. A random forest [7] with 100 trees is trained on single flows with the same features and the same split, built by the same code, and scored on the last flow of each CNN-LSTM test window. Both models are thus judged on identical examples with identical labels, and any difference between them is due to the model, not the data.

3.2.5 Model Development

Architecture. The model, described in detail in Section 4.5.3, consists of two one-dimensional convolution blocks that learn local patterns across neighbouring flows, an LSTM layer that summarises how these patterns develop across the window, a dense layer and a softmax output. The same architecture is used for both datasets; only the input shape and the number of output classes differ.

Class imbalance. Benign traffic is 83% of the CICIDS2017 test set and some attack classes have only a few hundred examples, so the training loss weights each class (Section 4.5.3). Three weighting schemes were trained with the same architecture and scored on the same test set (Table 3.4). Capped balanced weights pushed benign traffic into rare classes, and a targeted boost of the three weakest classes made the overall result worse; the square root of the balanced weights gave the best result and is the deployed model (v2).

Table 3.4. Class-weighting schemes compared on the CICIDS2017 test set

Version	Class weights	Test accuracy	Macro F1
v1	Balanced, capped at 50	0.8831	0.4276
v2 (deployed)	**Square root of balanced, no cap**	**0.9842**	**0.5857**
v3	Three-times boost on the three weakest classes	0.9470	0.4811

Training procedure. Both models use the Adam optimiser, sparse categorical cross-entropy, a batch size of 512 and early stopping on the validation loss with a patience of five epochs, restoring the best weights. The CICIDS2017 model used a stratified 10% validation split; its lowest validation loss came in the first epoch (0.153, validation accuracy 0.954), so training stopped after the sixth epoch and the first-epoch weights were kept. The MU-IoT model was trained in mixed precision on a Colab T4 GPU; every epoch draws a fresh sample of up to 300,000 training windows per class, with square-root-balanced class weights capped at 10, and checkpoints are written every epoch so that a disconnected session loses at most one epoch. Early stopping ended training at epoch 12 and kept the weights of epoch 7.

3.2.6 Explainability Method

Two SHAP estimators are used, each chosen for its purpose.

Per-prediction explanations in the dashboard. The dashboard explains the selected window with Kernel SHAP [15], which treats the model as a black box, with 1,000 samples, attributions grouped per feature across the ten time steps (20 values instead of 200), a fixed random seed and a background of real training windows. The explainer and its configuration were chosen by measurement against the gradient-based alternative (Section 6.3.3).

Error analysis of the MU-IoT model. The offline analysis uses expected gradients [15], with 500 samples per window and a background of 350 real training windows (50 per class). For each type of error, the mean SHAP profile of the wrongly classified windows is compared with the profiles of correctly classified windows of the predicted and of the true class, which shows whether an error is a systematic resemblance, a real similarity between attacks, or an extrapolation beyond the training data.

3.2.7 Prevention Policy Design

Each class is mapped to a countermeasure (block the source address, drop its connections for 60 seconds, rate-limit it for 120 seconds, isolate the host, or a recommendation only), a severity and a confidence threshold. The threshold rises as the class's measured F1 score falls, so reliable classes act at lower confidence and unreliable ones need more. Classes whose evidence is too weak are locked to human review whatever the confidence: on CICIDS2017, SQL injection, Heartbleed and Infiltration have too few test examples to measure, and Bot, Web Attack - Brute Force and XSS have an F1 score below 0.15; on MU-IoT, MiTM and Spyware have no held-out recording. Approved actions are executed as Windows Firewall rules by an execution engine with independent safety checks. The policy, the decision rule and the execution engine are described in Section 4.5.6 and Section 4.5.7, and the policy's measured effect in Section 6.3.4.

3.2.8 Live Capture and Packet-to-Flow Extraction

The model was trained on flows produced by CICFlowMeter version 3. To apply it to live traffic or to Wireshark captures, packets must be turned into flows with exactly the same definitions, including the tool's quirks, because the model learned the features as the tool computed them. A new extractor was written for this purpose and validated against the original CICIDS2017 capture (Section 6.3.5). Table 3.5 lists the definitions it reproduces.

Table 3.5. CICFlowMeter-V3 behaviour reproduced by the extractor

Behaviour	Effect on the features
First packet counted twice	Packet Length Mean, Std and Variance and Average Packet Size include the flow's first packet twice
Sample standard deviation	Standard deviations divide by n - 1
Ethernet padding counted as payload	Frames shorter than Ethernet's 60-byte

Behaviour	Effect on the features
	minimum are padded, as on the wired testbed
Whole-microsecond timestamps	Times and inter-arrival times are integers in microseconds
Flow ends on FIN	A FIN ends an existing flow; a FIN that starts a new flow does not (the "TCP appendix" [18])
120-second flow timeout	A flow ends 120 seconds after it started
Subflows	Subflow values equal the flow totals

The extractor reads IPv4 and IPv6 packets directly from raw bytes. Live mode differs from CICFlowMeter-V3 in one deliberate way: it also ends a flow on RST. A scan flow (SYN followed by RST) has the same features either way, but ending it at the RST reports the scan within seconds instead of after the 120-second timeout. The live engine is described in Sections 4.4 and 4.5.

3.2.9 Dashboard and Backend

A Flask backend loads the model, the scaler and the SHAP explainer and exposes them through a REST API (Appendix A). A React and TypeScript dashboard offers two modes: batch analysis of an uploaded flow CSV file or Wireshark capture, and a live feed that either replays a file or captures from a network interface. For display, the dashboard maps the fifteen CICIDS2017 classes to nine categories that follow CERT-In's grouping of incidents (Appendix B). Every prediction shows its class, confidence, prevention decision and, on request, its SHAP explanation; the dashboard distinguishes an action that was only recommended from one that was actually executed, and lists the blocked devices with a revoke control.

3.2.10 Evaluation Metrics

For each class k, precision P and recall R are computed from the true positives (TP), false positives (FP) and false negatives (FN) of that class:

$$P_k = \frac{TP_k}{TP_k + FP_k}, \qquad R_k = \frac{TP_k}{TP_k + FN_k} \tag{3.3}$$

The F1 score is their harmonic mean:

$$F1_k = \frac{2\,P_k\,R_k}{P_k + R_k} \tag{3.4}$$

Two averages over the K classes are reported. The macro F1 gives every class the same weight and therefore exposes failures on rare classes; the weighted F1 weights each class by its number of examples N_k:

$$F1_{macro} = \frac{1}{K}\sum_{k=1}^{K} F1_k, \qquad F1_{weighted} = \sum_{k=1}^{K} \frac{N_k}{N} F1_k \tag{3.5}$$

Accuracy is the fraction of correctly classified examples:

$$\text{Accuracy} = \frac{1}{N}\sum_{n=1}^{N} \mathbf{1}[\hat{y}_n = y_n] \tag{3.6}$$

Because benign traffic dominates both datasets, accuracy and weighted F1 stay high however the rare attacks fare, so macro F1 and the per-class F1 scores are the main measures of detection quality in this report.

Chapter 4

System Design

This chapter presents the design of the system: its architecture (Section 4.1), the use cases it supports (Section 4.2), the flow of data through it (Section 4.3), the interactions between its components over time (Section 4.4) and the algorithms it uses (Section 4.5).

4.1 System Architecture

The system has three layers (Fig. 4.1). The presentation layer is the dashboard in the user's web browser. The application layer is a Flask backend that runs on the protected computer and contains the processing pipeline: input sources, the packet-to-flow extractor, the window builder, the CNN-LSTM classifier, the prevention policy, target selection, the prevention engine and the SHAP service. The host and data layer holds the trained artifacts, the prevention log and the Windows Firewall, which actually enforces the blocks.

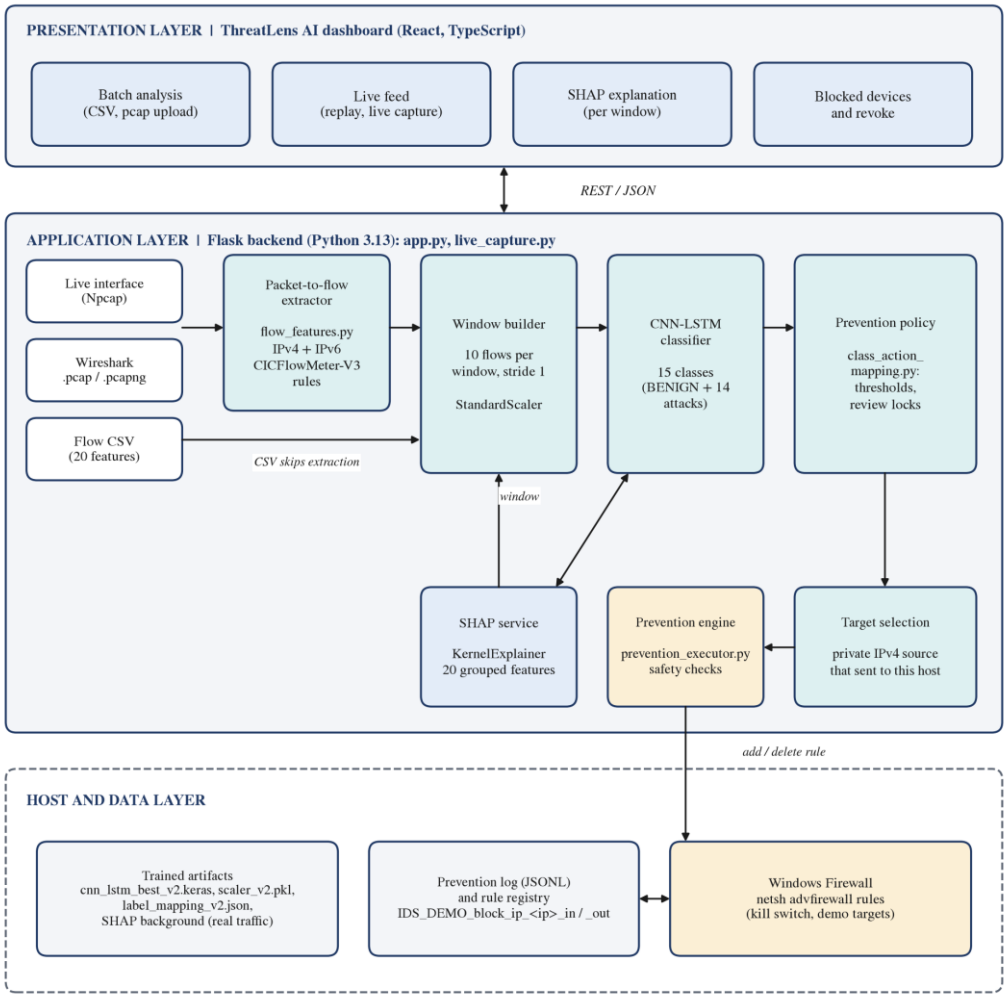


Fig. 4.1. System architecture

Data enters the application layer from one of three sources. A live network interface delivers packets through the Npcap driver, and an uploaded Wireshark capture (.pcap or .pcapng) delivers recorded packets; both go through the packet-to-flow extractor, which produces one row of 20 features per flow. An uploaded flow CSV already contains the features and goes straight to the window builder. From there every source follows the same path: windows of ten flows are scaled and classified, the predicted class and confidence are checked against the prevention policy, a target address is selected, and the prevention engine adds or refuses a firewall rule. The SHAP service explains any window on request by querying the model repeatedly. The dashboard talks to the backend only through a REST API with JSON messages.

Table 4.1 maps each component of Fig. 4.1 to the module that implements it and the team member responsible for it.

Table 4.1. Components of the system and their implementation

Component	Module	Responsibility
Dashboard (batch analysis, live feed, SHAP view, blocked devices)	frontend/src/routes/dashboard.tsx, components/LiveFeed.tsx	Varshini D N
REST API, upload handling, CERT-In mapping	backend/app.py, backend/cert_in_mapping.py	Varshini D N
SHAP service	backend/shap_service.py	Varshini D N
Data cleaning, feature selection, MU-IoT handoff	src/ scripts, output/ metadata	Maanya B S
Packet-to-flow extractor	model/flow_features.py	Ruthwik Sai Ganesh
Live capture engine	backend/live_capture.py	Ruthwik Sai Ganesh
CNN-LSTM model and training	model/cnn_lstm_architecture.py, model/train_cnn_lstm.py	Ruthwik Sai Ganesh
Prevention policy	model/class_action_mapping.py	Ruthwik Sai Ganesh
Target selection	model/target_selection.py	Ruthwik Sai Ganesh
Prevention engine	model/prevention_executor.py, backend/prevention_runtime.py	Ruthwik Sai Ganesh

Three design decisions shape the architecture.

1. **One pipeline for all inputs.** Live traffic, Wireshark captures and flow CSV files are converted to the same 20 features and go through the same model and policy, so a result obtained on a capture file carries over to live traffic.
2. **Blocking does not depend on the dashboard.** The live engine calls the prevention engine itself. A scan is blocked even if no browser is open, and the dashboard only reports what happened.
3. **Safety is enforced in the engine, not in the user interface.** The kill switch can only be turned on through an environment variable when the backend is started; no API request can enable execution or widen the set of addresses that may be blocked.

4.2 Use Case Diagram

Fig. 4.2 shows the use cases of the system. The primary actor is the security analyst who operates the dashboard. Two secondary actors take part: the network traffic source, which produces the packets that are analysed (and may be an attacker), and the Windows Firewall, which enforces and removes blocks.

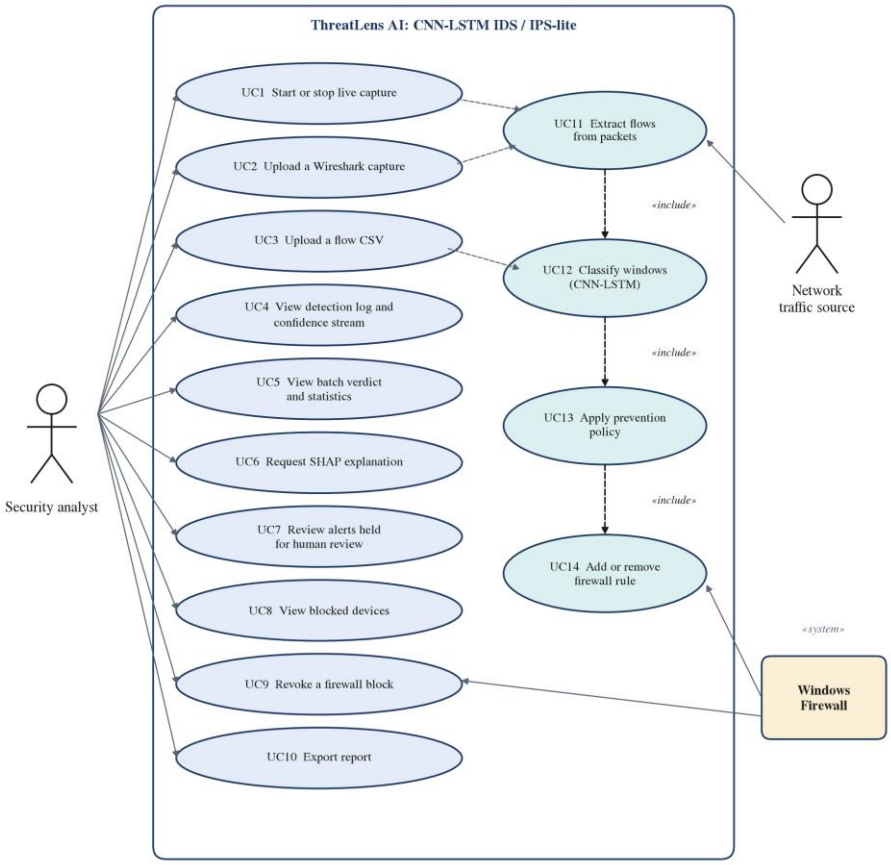


Fig. 4.2. Use case diagram

The analyst's ten use cases (UC1 to UC10) cover controlling the input, viewing results and managing blocks. Four system use cases (UC11 to UC14) are included by them: starting a capture or uploading a Wireshark file includes extracting flows from packets; extracting flows, or uploading a flow CSV, includes classifying windows; classification includes applying the prevention policy; and the policy includes adding a firewall rule when an automatic action is approved. Revoking a block (UC9) removes a rule.

4.3 Data Flow Diagram

The data flow diagrams show what data the system consumes and produces, without the timing of the interactions. At level 0 (Fig. 4.3(a)) the whole system is a single process. It receives live packets from the network interface and capture files or flow CSV files from the analyst, receives the analyst's requests (start or stop capture, upload, explain, revoke), returns verdicts, SHAP values and the list of blocked devices, and exchanges block and unblock rules and their status with the Windows Firewall.

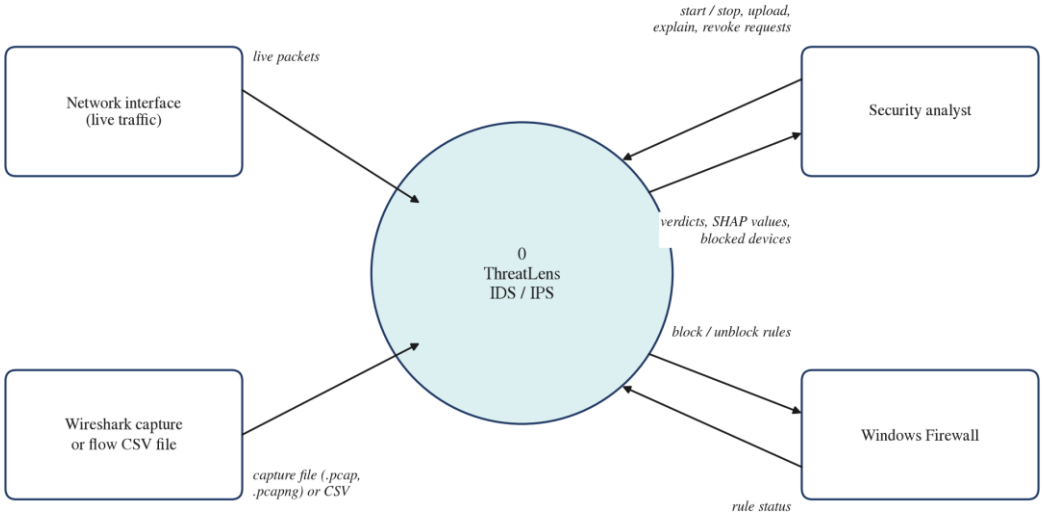


Fig. 4.3(a). Data flow diagram, level 0

Level 1 (Fig. 4.3(b)) decomposes the system into seven processes and four data stores.

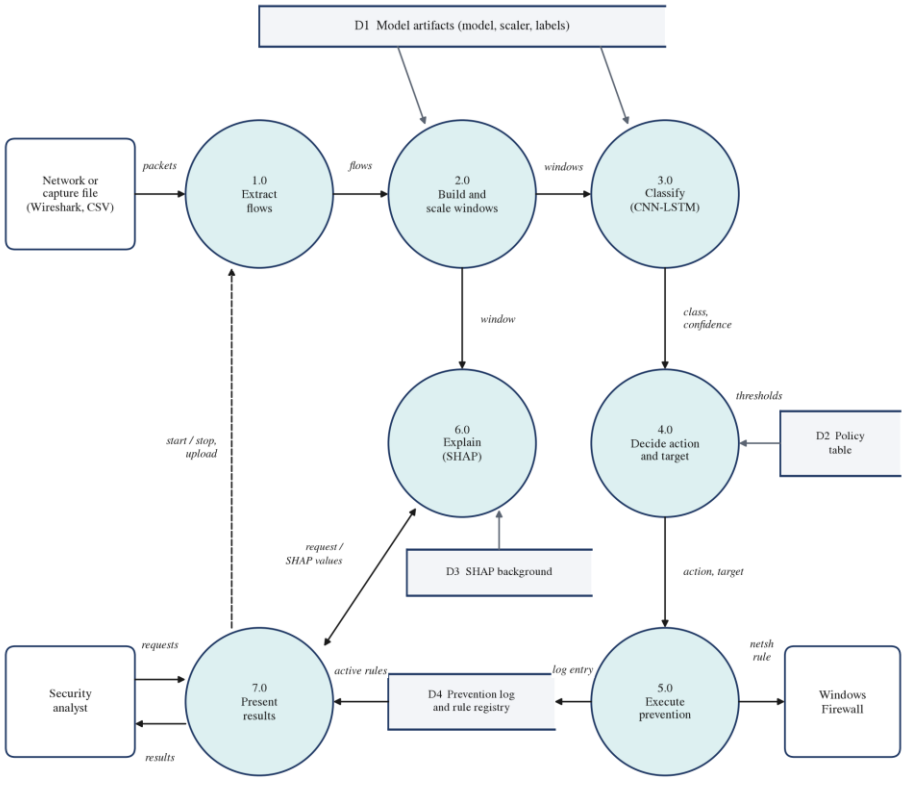


Fig. 4.3(b). Data flow diagram, level 1

The analyst never writes to the policy table or the engine's settings; requests reach process 5.0 only as revocations, and every attempt to act, including refusals, is written to the prevention log (D4).

4.4 Sequence Diagram

The sequence diagrams show the order of the interactions between the components for the four main scenarios.

Live detection and blocking (Fig. 4.4(a)). After the analyst starts a capture (messages 1 and 2), the live engine builds flows from the captured packets (3). Once per second it classifies the ten most recently ended flows (4 and 5), asks the policy for the action (6 and 7), selects the target (8) and calls the prevention engine (9), which runs its safety checks (10) before adding the firewall rules (11 and 12). The dashboard collects the results by polling once per second (14 and 15); blocking itself does not wait for it.

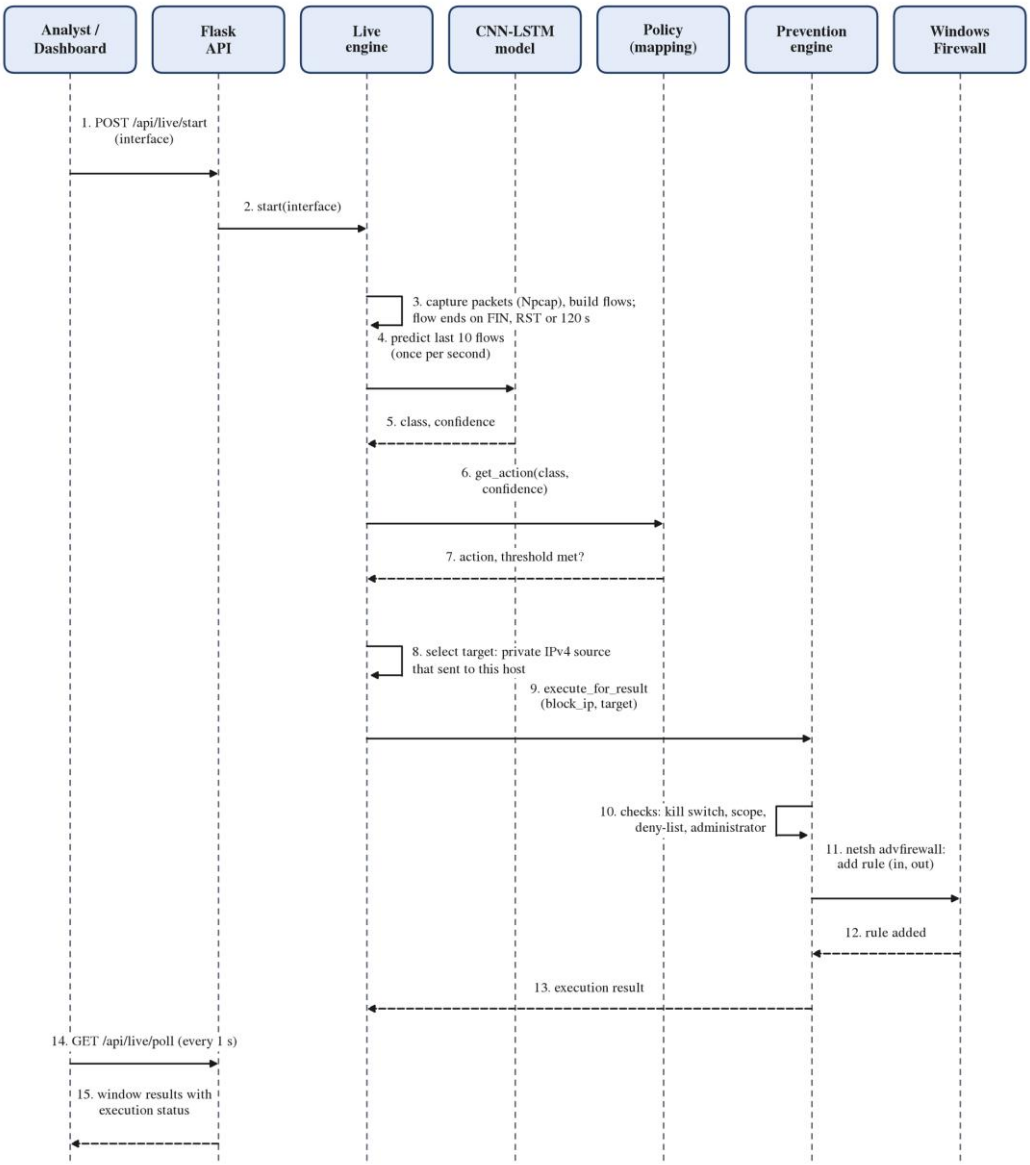


Fig. 4.4(a). Sequence diagram: live detection and automatic blocking

Upload of a Wireshark capture (Fig. 4.4(b)). The backend checks the file (2), extracts flows (3 and 4), builds windows (5), and classifies every window (6 and 7) and passes it through the policy (8 and 9); executions are attempted only within the allowed scope and with the kill switch on (10 and 11). A flow CSV skips steps 2 to 4.

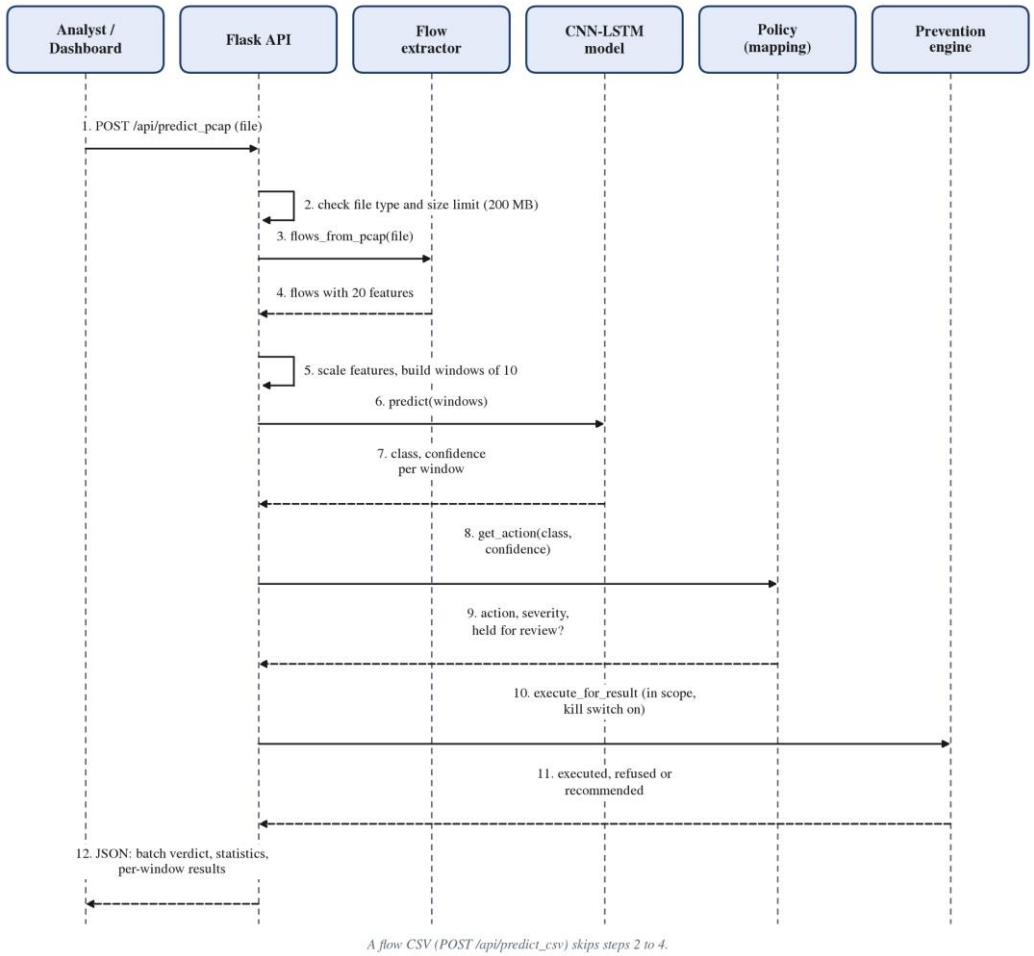


Fig. 4.4(b). Sequence diagram: analysis of an uploaded capture

SHAP explanation (Fig. 4.4(c)). The SHAP service groups the selected window's 200 inputs into 20 features (4), queries the model on 1,000 perturbed samples built from a real-traffic background (5 and 6) and solves Kernel SHAP's weighted regression, checking that the attributions add up to the model's output (7).

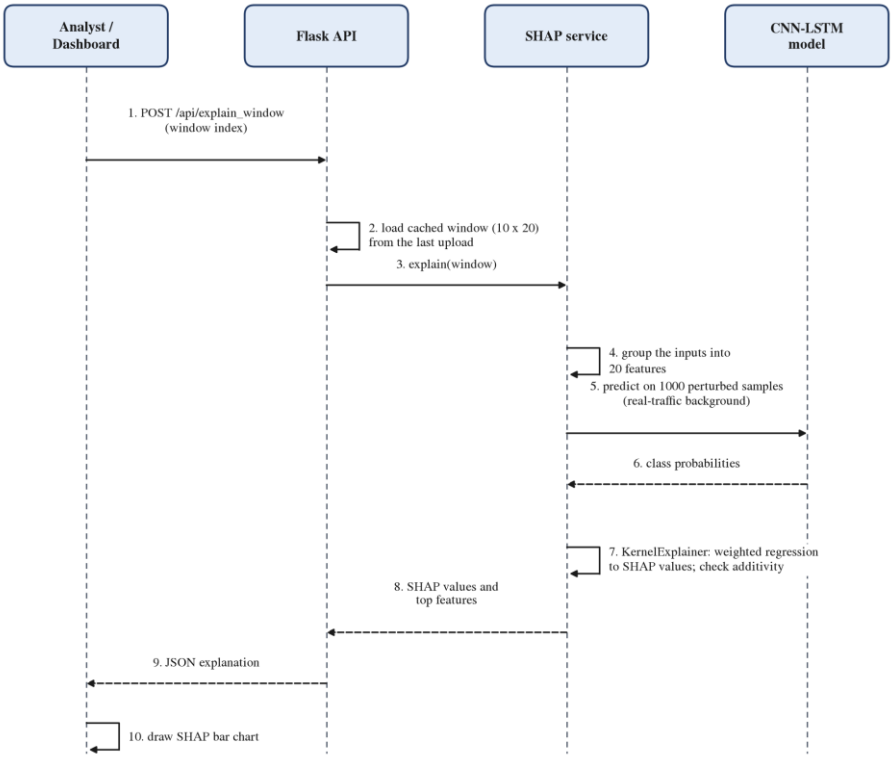


Fig. 4.4(c). Sequence diagram: SHAP explanation of a window

Revocation of a block (Fig. 4.4(d)). Revocation checks only the target's scope and administrator rights; the kill switch is not required, so a block can always be undone (7).

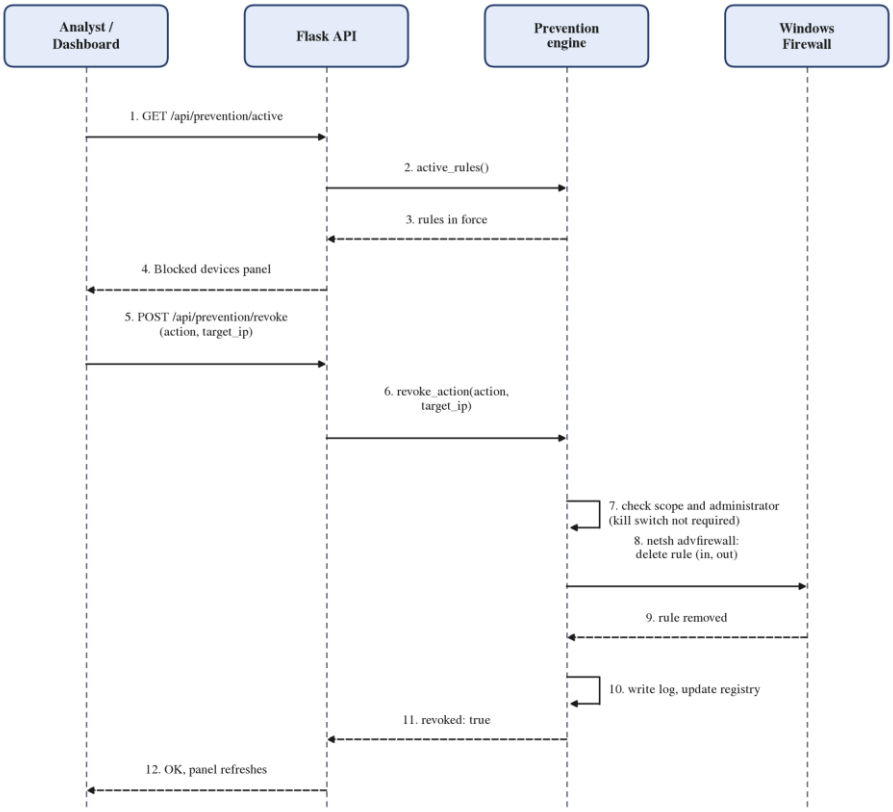


Fig. 4.4(d). Sequence diagram: revoking a firewall block

4.5 Algorithms Used

4.5.1 Packet-to-Flow Extraction

The extractor groups packets into bidirectional flows identified by the five-tuple (source and destination address and port, protocol); the direction of a flow's first packet is its forward direction. For each flow it keeps the length L and timestamp t of every packet in each direction. When the flow ends, the 20 features are computed with CICFlowMeter-V3's definitions. Packet-length statistics count the first packet L_1 twice, and standard deviations are sample standard deviations, so for a flow of n packets:

$$\bar{L} = \frac{1}{n+1}\left(L_1 + \sum_{i=1}^{n} L_i\right) \tag{4.1}$$

$$s_L = \sqrt{\frac{1}{n}\left[(L_1 - \bar{L})^2 + \sum_{i=1}^{n}(L_i - \bar{L})^2\right]} \tag{4.2}$$

$$\text{AvgPacketSize} = \frac{1}{n}\left(L_1 + \sum_{i=1}^{n} L_i\right) \tag{4.3}$$

Inter-arrival times are differences between consecutive packet timestamps, in whole microseconds:

$$\mathrm{IAT}_i = t_{i+1} - t_i, \qquad \mathrm{FlowIATMax} = \max_i \ \mathrm{IAT}_i \tag{4.4}$$

Algorithm 4.1. Packet-to-flow extraction (model/flow_features.py)

```
for each packet p (from Npcap or a capture file):
    parse IPv4 or IPv6 header, skip IPv6 extension headers; keep TCP and
UDP
    length := transport payload bytes, padded to Ethernet's 60-byte minimum
    key := (src, sport, dst, dport, proto), direction-independent
    if key not in table:
        create flow; forward := direction of p
        add p; continue                  # a FIN that starts a flow does
not end it
    flow := table[key]
    if p.time - flow.start > 120 s:
        emit features(flow); create a new flow with p; continue
    add p to flow (forward or backward)
    if p has FIN, or (live mode and p has RST):
        emit features(flow); remove flow from table
at end of capture: emit features of every open flow
```

4.5.2 Window Construction and Scaling

Each flow's 20 features are standardised with Equation (3.1), using the mean and standard deviation of the training rows, and windows of ten consecutive flows are formed as in Equation (3.2). In training and evaluation the window slides by one flow. In live operation the engine classifies, once per second, the window made of the ten most recently ended flows.

4.5.3 Hybrid CNN-LSTM Classifier

The classifier (Fig. 4.5, Table 4.2) maps a window of 10 x 20 values to a probability for each of the 15 classes.

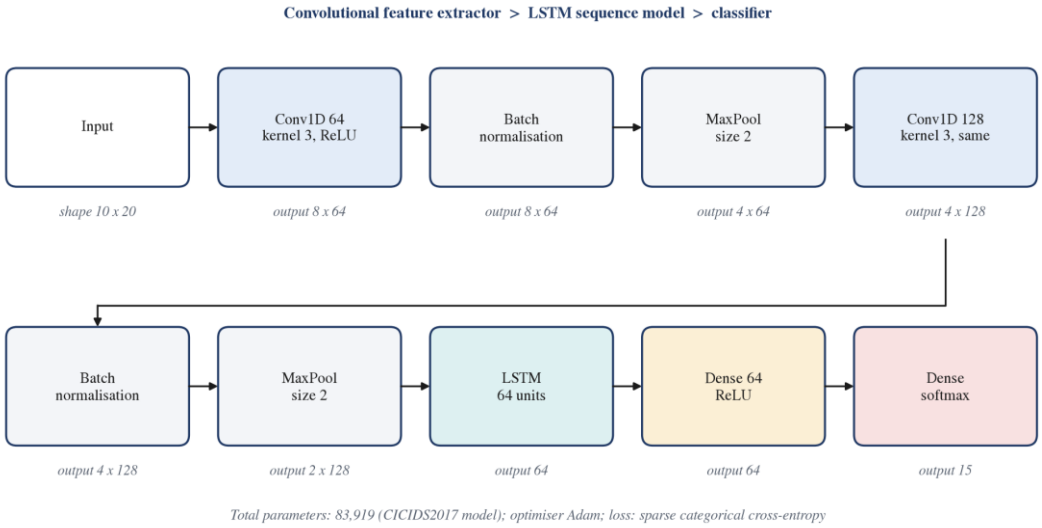


Fig. 4.5. Architecture of the hybrid CNN-LSTM model with the output shape of each layer

Table 4.2. Layers of the CNN-LSTM models

Layer	**CICIDS2017 model**	**MU-IoT model**
Input	10 flows x 20 features	20 flows x 38 features
Conv1D, 64 filters, kernel 3, ReLU, no padding; BatchNorm; MaxPool 2	Yes	Yes
Conv1D, 128 filters, kernel 3, ReLU, same padding; BatchNorm; MaxPool 2	Yes	Yes
LSTM, 64 units	Yes	Yes
Dense, 64 units, ReLU	Yes	Yes
Dense, softmax	15 classes	7 classes
Trainable and non-trainable parameters	83,919	86,855
Numerical precision	float32	mixed float16 (float32 output)

Convolution. A one-dimensional convolution slides 64 filters of width three along the ten flows. Each filter k combines three neighbouring flows t, t+1, t+2 over all C input features with weights w and a bias b, followed by the rectified linear unit:

$$h_{t,k} = \mathrm{ReLU}\left(b_k + \sum_{m=0}^{2} \sum_{c=1}^{C} w_{k,m,c}\, x_{t+m,\,c} \right), \qquad \mathrm{ReLU}(a) = \max(0, a) \tag{4.5}$$

Batch normalisation and pooling. Batch normalisation rescales each filter's output with the mean μ and variance σ² of the mini-batch and two learned

parameters γ and β, which stabilises training; max pooling keeps the larger of each pair of neighbouring values and halves the length of the sequence:

$$\hat{h} = \gamma \frac{h - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} + \beta \tag{4.6}$$

$$p_{t,k} = \max\left(h_{2t,\,k},\ h_{2t+1,\,k}\right) \tag{4.7}$$

The second block repeats these operations with 128 filters and same padding. After both blocks the ten flows have become a sequence of two steps of 128 learned features.

LSTM. The LSTM [9] reads this sequence step by step. At each step t its forget gate f, input gate i and output gate o, computed from the current input x and the previous hidden state h with weight matrices W and U and biases b, decide what the memory cell c forgets, stores and exposes:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \quad i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \quad o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \tag{4.8}$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c), \qquad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{4.9}$$

$$h_t = o_t \odot \tanh(c_t) \tag{4.10}$$

where σ is the logistic sigmoid and ⊙ is element-wise multiplication. The final hidden state h, of 64 values, summarises the whole window.

Output. A dense layer of 64 units with ReLU is followed by a softmax layer that turns the K output scores z into class probabilities. The predicted class is the one with the largest probability, and that probability is the prediction's confidence c:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad c = \max_k\ \hat{y}_k \tag{4.11}$$

Training objective. The model minimises the class-weighted sparse categorical cross-entropy over N training windows with true classes y:

$$\mathcal{L} = -\frac{1}{N} \sum_{n=1}^{N} w_{y_n} \log \hat{y}_{n,\,y_n} \tag{4.12}$$

with the deployed square-root-balanced class weights, where N_k is the number of training windows of class k:

$$w_k = \sqrt{\frac{N}{K N_k}} \tag{4.13}$$

Square-root weights give rare classes more influence than they would have otherwise without letting them dominate: a class one hundred times rarer than another is weighted ten times more, not one hundred times.

4.5.4 Random-Forest Baseline

The baseline is a random forest [7] of 100 decision trees with a fixed random seed of 42, trained on single flows with the same features and split. Each tree is grown on a bootstrap sample of the training data, and at each split considers a random subset of the features; the forest predicts the class chosen by most trees. It is scored on the last flow of each CNN-LSTM test window, so both models see the same test examples.

4.5.5 SHAP Explanations

SHAP [15] explains a prediction f(x) by giving every feature i a value φ_i, its Shapley value: the average change in the model's output when feature i is added to a subset S of the other features, over all subsets F, weighted by how many orderings produce each subset:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\ (|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)] \tag{4.14}$$

The values satisfy local accuracy: together with the base value φ_0, the model's average output over the background data, they add up exactly to the prediction being explained:

$$f(\mathbf{x}) = \phi_0 + \sum_{i=1}^{M} \phi_i \tag{4.15}$$

Computing Equation (4.14) exactly requires the model's output for every subset of features, which is infeasible for 20 features. Kernel SHAP estimates the values by sampling subsets (coalitions) z of the M features, replacing absent features with values from the background data, and fitting a weighted linear regression with the SHAP kernel weight:

$$\pi(\mathbf{z}) = \frac{M - 1}{\binom{M}{|\mathbf{z}|}\ |\mathbf{z}|\ (M - |\mathbf{z}|)} \tag{4.16}$$

The dashboard uses Kernel SHAP with M = 20 feature groups, 1,000 sampled coalitions and a background of 100 real training windows. The offline error analysis uses expected gradients, which average the gradient of the model along straight paths from background samples x' to the input x:

$$\phi_i = \mathbb{E}_{\mathbf{x}' \sim D,\ \alpha \sim U(0,1)} \left[(x_i - x'_i)\ \frac{\partial f(\mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}'))}{\partial x_i} \right] \tag{4.17}$$

and compares the mean SHAP profiles a and b of two groups of windows with the cosine similarity:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\ \|\mathbf{b}\|} \tag{4.18}$$

4.5.6 Prevention Policy

The policy table (Table 4.3) assigns every class an action, a severity and a confidence threshold. Thresholds were calibrated inversely to each class's F1 score on the test set, through a decreasing piecewise-linear curve: the CICIDS2017 classes define the calibration points ($F1_j$, θ_j), and the threshold of any other F1 value, such as those of the MU-IoT classes, is interpolated between the two nearest points:

$$\theta(F1) = \theta_j + \frac{F1 - F1_j}{F1_{j+1} - F1_j}(\theta_{j+1} - \theta_j), \qquad F1_j \le F1 \le F1_{j+1} \tag{4.19}$$

Table 4.3. Prevention policy for the CICIDS2017 classes

Class	Action	Severity	F1	Threshold	Automatic
BENIGN	No action	Low	0.991	0.50	-
DDoS	Block IP	Critical	0.990	0.55	Yes
DoS Hulk	Rate limit (120 s)	High	0.990	0.55	Yes
PortScan	Block IP	Medium	0.971	0.60	Yes
DoS Slowhttptest	Drop connection (60 s)	Medium	0.886	0.65	Yes
DoS GoldenEye	Rate limit (120 s)	High	0.816	0.70	Yes
FTP-Patator	Block IP	High	0.773	0.72	Yes
DoS slowloris	Drop connection (60 s)	Medium	0.725	0.75	Yes
SSH-Patator	Block IP	High	0.6	0.78	Yes

Class	Action	Severity	F1	Threshold	Automatic
			56		
Web Attack - Brute Force	Block IP	High	0.125	0.92	No, review
Bot	Isolate host	High	0.056	0.94	No, review
Web Attack - XSS	Sanitise input	High	0.006	0.97	No, review
Web Attack - Sql Injection	Block IP	Critical	-	-	No, review
Heartbleed	Terminate session	Critical	-	-	No, review
Infiltration	Isolate host	Critical	-	-	No, review

For MU-IoT the same curve gives thresholds of 0.63 for Injection, 0.71 for Password_Hacking and 0.84 for DDoS and Scan, from the F1 scores on the held-out recordings; every automatic MU-IoT action is a block of the source address, and MiTM and Spyware are locked to review. A prediction $\hat{y}$ with confidence c triggers an automatic action only if it is not benign, its class is not in the set $\mathcal{R}$ of classes locked to review, and its confidence reaches the class's threshold:

$$\text{auto}(\hat{y}, c) = \mathbf{1}[\hat{y} \neq \text{benign}] \cdot \mathbf{1}[\hat{y} \notin \mathcal{R}] \cdot \mathbf{1}[c \geq \theta_{\hat{y}}] \tag{4.20}$$

Otherwise the prediction is held for review (locked class or confidence below threshold) or needs no action (benign). The decision procedure, including target selection and the execution checks, is shown as a flowchart in Fig. 4.6.

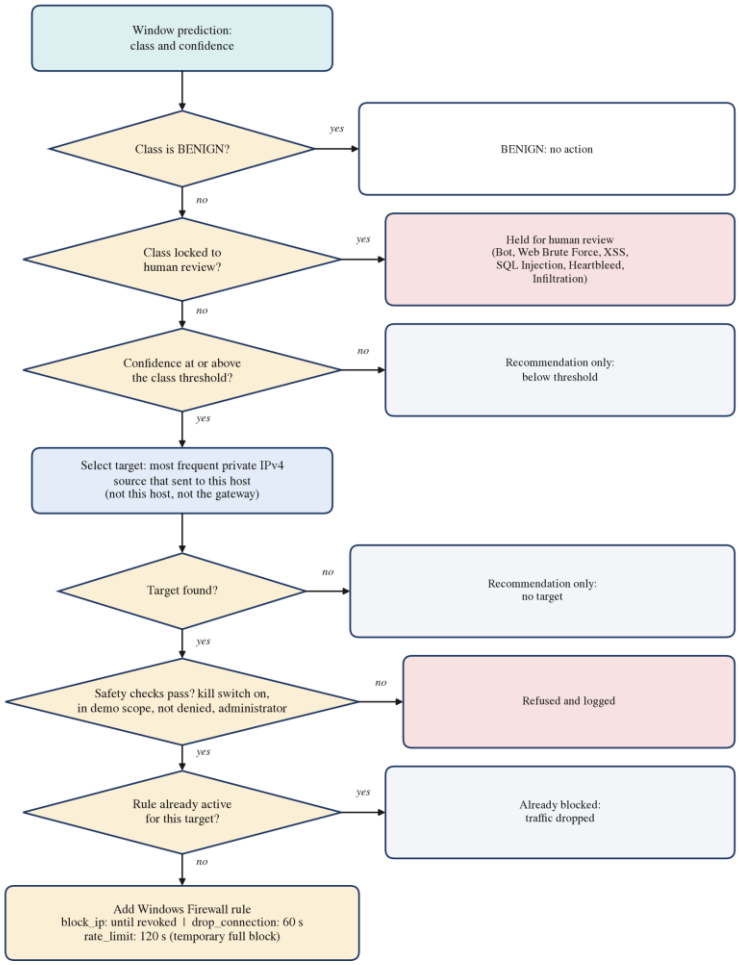


Fig. 4.6. Flowchart of the prevention decision for one window

4.5.7 Target Selection and Firewall Execution

Target selection. A window contains ten flows that may come from several devices. The target is the most frequent source address a among the eligible sources A of the window W:

$$a^{*} = \arg\max_{a \in A} \left| \{ f \in W : \mathrm{src}(f) = a \} \right| \tag{4.21}$$

A source is eligible only if it is a private (local network) IPv4 unicast address, is neither the protected computer itself nor its gateway, and has sent traffic to the protected computer. The last condition was added after the live run (Section 6.3.7): a device that only broadcasts discovery messages to the whole network never addresses the protected computer and must never be blocked by it. If no source is eligible, the action stays a recommendation.

Execution. The prevention engine turns an approved action into two Windows Firewall rules, inbound and outbound, named after the action and the address (for example `IDS_DEMO_block_ip_<address>_in`), through `netsh advfirewall`. Algorithm 4.2 shows the checks it applies.

Algorithm 4.2. Execution of a prevention action (model/prevention_executor.py)

```
execute(action, target, class, confidence):
    if kill switch is off:                        refuse "execution
disabled"
    if target not a valid single IPv4 address:    refuse
    if target in deny-list (loopback, broadcast, unspecified): refuse
    if target not in RFC 5737 TEST-NET ranges and
       target not listed in IDS_PREVENTION_DEMO_TARGETS:      refuse "out
of scope"
    if not running as administrator:              refuse
    if a rule for target is already active:       return "already blocked"
    add inbound and outbound block rules (netsh advfirewall)
    if action is drop_connection or rate_limit:   start expiry timer (60 s /
120 s)
    log the attempt and its outcome (JSONL)

revoke(action, target):                           # kill switch not required
    if target out of scope or not administrator: refuse
    delete both rules; cancel timer; log the revocation
```

Only one rule is kept per device, whatever class flags it and however many windows do, and a startup sweep restores the engine's view of the firewall after a crash. Rate limiting is implemented as a temporary full block, because Windows offers no simple per-address bandwidth limit, and input sanitisation against XSS remains a recommendation, because a network firewall cannot perform it.

Chapter 5

Hardware and Software Requirement

This chapter lists the hardware and software that were used to develop, train and run the system, and the minimum requirements for running it. The project was developed on ordinary student equipment: one laptop serves as both development machine and protected host, and model training was done on free cloud GPUs.

5.1 Hardware Requirement

Table 5.1 lists the hardware actually used. All speed measurements in this report were taken on the laptop listed, without a GPU.

Table 5.1. Hardware used in the project

Item	Specification	Use
Laptop (protected host)	AMD Ryzen 3 5300U, 4 cores / 8 threads, 7.3 GB RAM, Windows 11 Home, Wi-Fi	Development, backend, dashboard, live capture, firewall
Cloud GPU	Google Colab GPU runtime (NVIDIA T4 for the MU-IoT model)	Training, large evaluations, extractor validation
Second device (attacker)	Android tablet with Termux and nmap	Port scans and page requests in the live tests
Network	Phone hotspot (captures) and a home Wi-Fi router (live run)	Shared network for the protected host and the second device

Training was done in the cloud because the full datasets do not fit comfortably in the laptop's memory: the MU-IoT feature matrix alone has almost four million rows, and training draws up to 300,000 windows per class per epoch. Inference, by contrast, is light: one window takes 1.48 ms on the laptop's CPU, so a GPU is not needed to run the system.

Table 5.2 gives the minimum requirements for running the trained system on one protected host.

Table 5.2. Minimum hardware requirements for running the system

Component	Minimum	Reason
Processor	64-bit dual-core x86 CPU	TensorFlow inference and the three capture threads
Memory	4 GB RAM (8 GB recommended)	Model, SHAP explainer and flow table in memory
Storage	2 GB free	Python environment, model

Component	Minimum	Reason
		artifacts, logs
Network interface	Wired or Wi-Fi adapter supported by Npcap	Live packet capture
Operating system	Windows 10 or 11, run as administrator	Npcap and Windows Firewall rules (netsh)

5.2 Software Requirement

Table 5.3 lists the software and libraries with the versions used. The backend's Python dependencies are pinned in `backend/requirements.txt`; SHAP is pinned exactly, because the explainer uses an internal SHAP class whose interface may change between versions.

Table 5.3. Software used in the project

Software	Version	Purpose
Windows 11 Home	10.0.26300	Operating system of the protected host
Python	3.13.2	Backend, model, extractor and tests
TensorFlow / Keras	2.21.0 / 3.15.0	CNN-LSTM training and inference
scikit-learn	1.6.1	Random forest, StandardScaler, metrics
NumPy / pandas	2.5.1 / 3.0.3	Numerical arrays and tables
SHAP	0.52.0	Kernel SHAP explanations
Scapy	2.8.0	Reading live packets (Npcap) and capture files
Flask, Flask-CORS	3.x, 4.x	REST API of the backend
Matplotlib	3.11	Figures and diagrams of this report
Node.js	24.22	Building and serving the dashboard
React / TypeScript	19.2 / 5.8	Dashboard user interface
Vite / TanStack Router	8.0 / 1.170	Dashboard build tool and routing
Recharts / Tailwind CSS	2.15 / 4.2	Charts and styling of the dashboard
Wireshark	4.6.9	Recording packet captures
Npcap	1.88	Packet capture driver for Wireshark and the live engine
nmap (Termux, Android)	Termux package	Port scans from the second device
Windows Firewall (netsh advfirewall)	Built in	Enforcing blocks
Google Colab	Cloud service	GPU training and large evaluations
Git and GitHub	-	Version control and collaboration between the three members

The backend must run with administrator rights, to capture packets and to change firewall rules, and no VPN may be active during live capture.

Chapter 6

Implementation, Testing and Results

This chapter describes how the system was implemented (Section 6.1) and tested (Section 6.2), reports all results (Section 6.3), discusses them (Section 6.4) and states the limitations of the work (Section 6.5). Every number in this chapter comes from a results file in the project repository; the files are named where the results are reported.

6.1 Implementation

6.1.1 Code Organisation

The project is kept in one Git repository shared by the three team members, with one branch per member and pull requests to merge finished work into the main branch. Data scripts are in `src/`, the model, evaluation, prevention and extractor code in `model/` (trained artifacts in `model/artifacts/`, every reported result in `model/results/`), the backend in `backend/` and the dashboard in `frontend/`. Two interface documents fix the contract between the members' work: the model interface states the order of the 20 features, the scaling, the window length and the label mapping, and the prevention interface states which actions exist and which checks the engine performs, so the model and the engine could change internally without breaking the dashboard.

6.1.2 Dashboard

The dashboard has a home page, an analysis console and an about page. The analysis console offers batch analysis and a live feed. In batch analysis (Fig. 6.1) the analyst uploads a flow CSV or a Wireshark capture and receives a batch verdict (attack rate, top threat, number of classes seen), network statistics (windows, normal and attack windows, actions recommended, actions actually executed and windows held for review), the class distribution, and a list of windows, each of which can be explained with SHAP. The header shows the loaded model with its test accuracy and weighted F1, whether the SHAP explainer is ready, and whether firewall execution is armed.

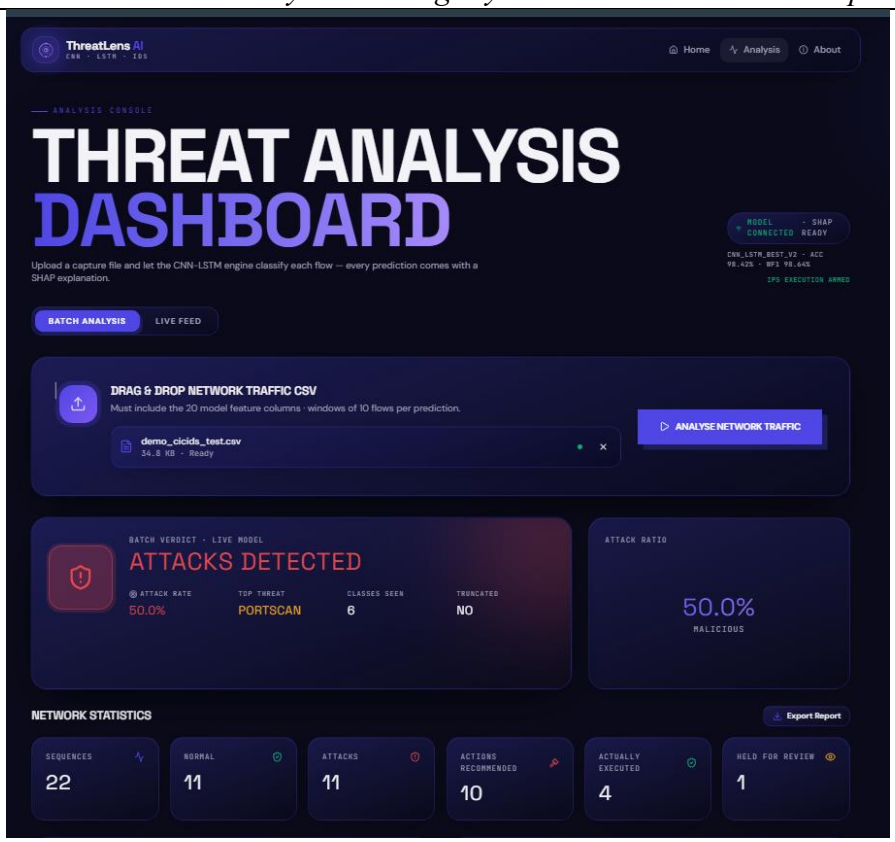


Fig. 6.1. Batch analysis of an uploaded flow file in the dashboard

The live feed either replays a file window by window or captures live traffic from a selected interface (Fig. 6.2). It shows the number of packets, flows and windows processed, the latest detection with its confidence and prevention status, a stream of the confidence of recent threat predictions, a detection log, and the Blocked devices panel. When real firewall execution is armed, a banner states which addresses may be acted on.

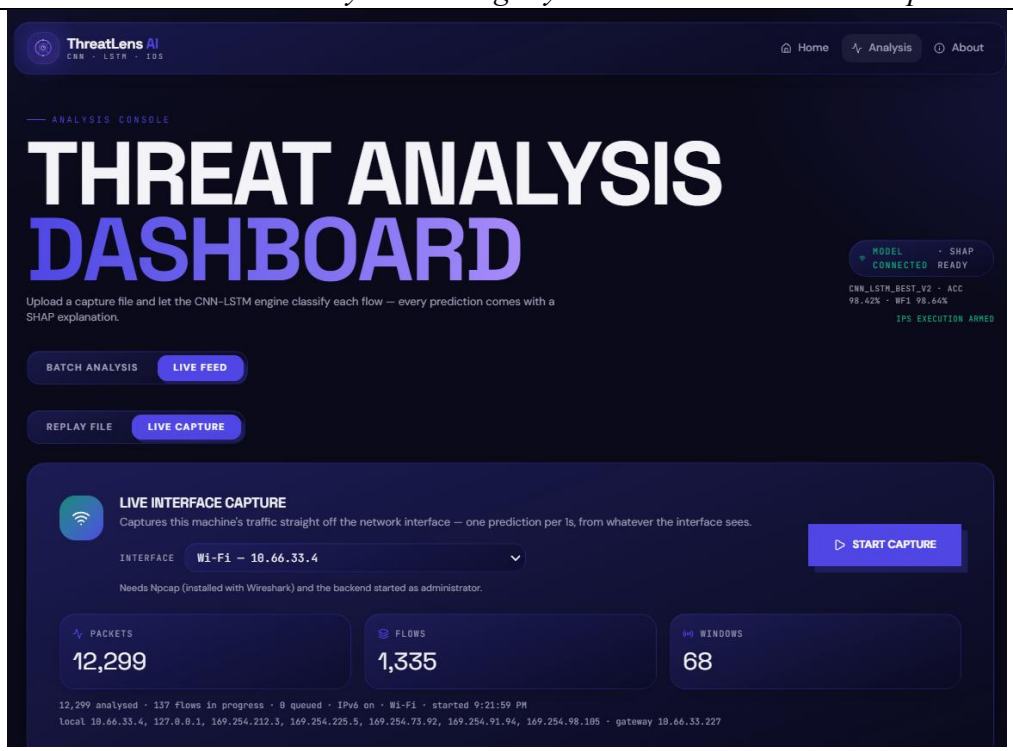


Fig. 6.2. Live interface capture in the dashboard during the final demonstration

6.1.3 Prevention in Operation

Figs. 6.3 to 6.5 show the prevention layer at work on live traffic. In Fig. 6.3, from the live run described in Section 6.3.7, window 6 was classified PortScan with a confidence of 0.997 and the scanning tablet was blocked; the following PortScan windows from the same device are reported as "already blocked, dropped" instead of creating new rules, and once the scan ended, the tablet's traffic returned to BENIGN.

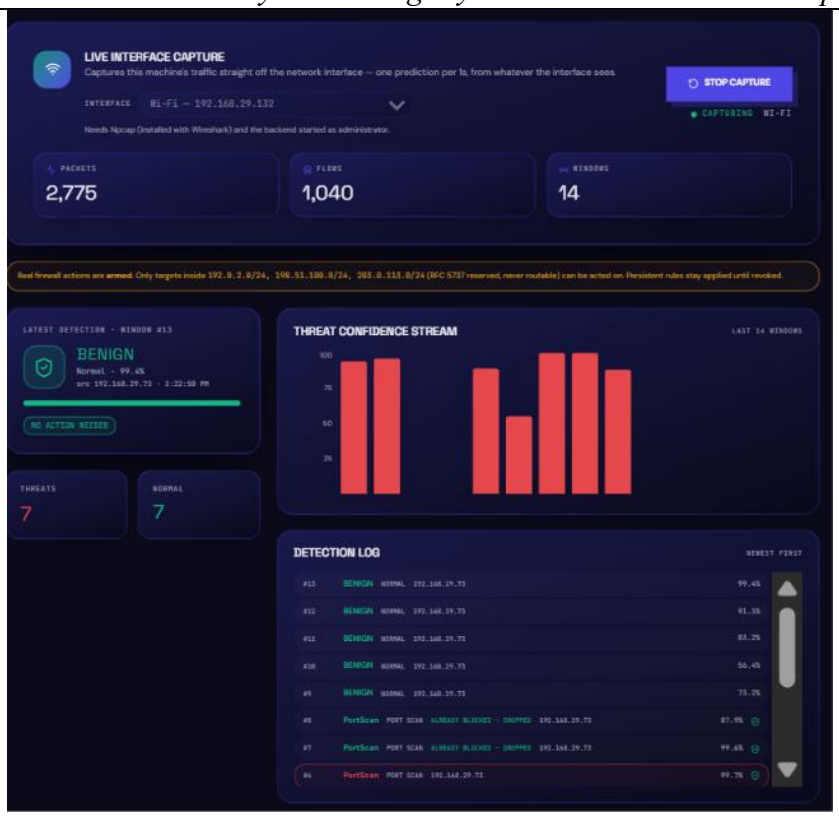


Fig. 6.3. Live run: a port scan detected and blocked, then reported as already blocked

Fig. 6.4 shows how the policy treats a class it does not trust. While the tablet repeatedly reloaded a page served by the laptop, the model classified several windows as Bot; Bot is locked to human review, so these windows were held for review (eye icon) and never executed, while the PortScan windows in the same log were acted on.

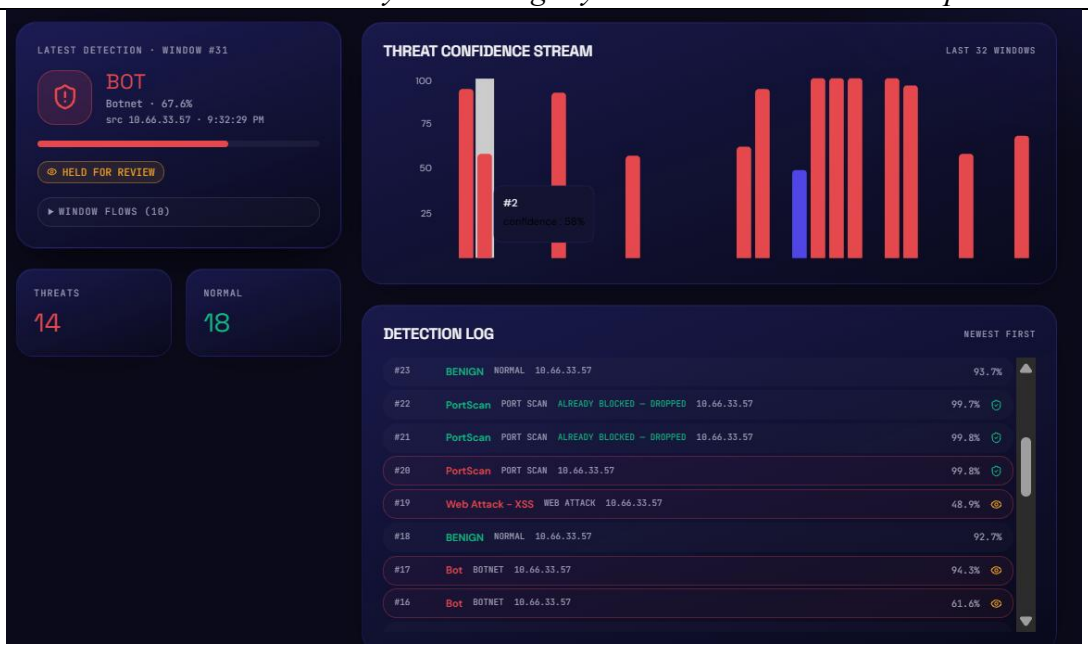


Fig. 6.4. Detection log: Bot predictions held for review, PortScan acted on

Fig. 6.5 shows the Blocked devices panel with the rule in force, its action (block IP, persistent until revoked), its firewall rule name and the revoke control; revoking the rule empties the panel and restores access.

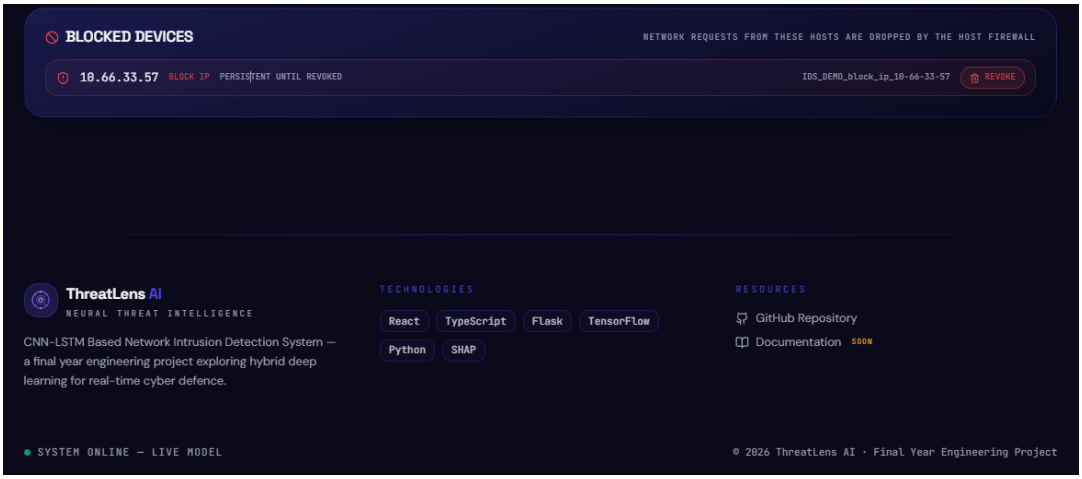


Fig. 6.5. Blocked devices panel with one rule in force and the revoke control

6.2 Testing

6.2.1 Automated Tests

Every component that can act on the network was tested automatically, without touching the real firewall, and then checked end to end on the real firewall. Table 6.1 lists the test suites.

Table 6.1. Automated tests

Suite	Tests	What it covers
model/test_prevention.py	35	Both policies, review locks, one rule per device, allow-list of listed devices, revoke, restart recovery, every refusal path
model/live_check_prevention.py	22 checks	End to end on the real Windows Firewall, with TEST-NET addresses only
model/test_flow_features.py	20	Feature definitions, CICFlowMeter-V3 quirks, IPv4 and IPv6 parsing, flow termination
backend/test_live_capture.py	14	Live engine threads, queue limits, prediction timing, polling, replay, without network, model or firewall
model/test_target_selection.py	8	Eligible sources, exclusion of this host, the gateway and broadcasting devices
model/test_mu_iot_windows.py	8	MU-IoT windows never cross a block, a split or a gap in row identifiers
model/test_validate_against_cicids.py	3	One-to-one flow matching, time-zone search, timestamps without AM/PM
backend api, stream and smoke tests	-	Dashboard endpoints and replay (reported as passing by the dashboard developer)

The prevention tests include adversarial targets, such as an address list written in netsh syntax (`192.0.2.10,8.8.8.8`), address ranges, IPv6 addresses and non-text input, all of which must be refused. The end-to-end check adds real rules once per target with no duplicates, has real public addresses refused, lets temporary rules expire on their timers, revokes with the kill switch off and recovers after a simulated restart; it passed 22 of 22 checks (`model/results/prevention_live_check.txt`) and removes every rule it created.

Mutation check. To make sure that the tests would notice a broken safety check, the prevention engine and policy were broken deliberately in 19 different ways (for example: kill switch removed, allow-list removed, a locked class unlocked, duplicate rules allowed, a stale timer allowed to remove a newer rule, the dataset argument ignored, benign traffic made actionable). For each breakage the test suite was run, and it failed every time: 19 of 19 breakages were caught.

6.2.2 Defects Found by Testing

Testing found defects that would otherwise have reached the live demonstration, all of which were fixed:

- revocations were logged without the name of the rule, and timer expiries were not logged at all;
- non-text targets were not explicitly refused;
- integrated with the backend, the engine added the same rule once for every flagged window, and refused to revoke whenever the kill switch was off;
- in the live run, a device that only broadcast discovery messages was chosen as a target, which led to the rule that a target must have sent traffic to the protected host (Section 6.3.7).

6.3 Results

6.3.1 Detection on CICIDS2017

Table 6.2 compares the CNN-LSTM with the random-forest baseline on the 530,951 identical test examples (`model/results/cnn_lstm_v2_full_test_results.txt`, `model/results/chronological_random_forest_results.txt`).

Table 6.2. CICIDS2017: CNN-LSTM and random forest on identical test examples

Model	Accuracy	Weighted F1	Macro F1
CNN-LSTM (10-flow windows)	0.9842	0.9864	0.5857
Random forest (single flows)	0.9911	0.9908	0.7803

Both models reach high accuracy and weighted F1, because benign traffic and the large attack classes dominate the test set. The macro F1, which weights every class equally, shows the difference: on CICIDS2017 the single-flow random forest is the better model. Table 6.3 gives the per-class scores.

Table 6.3. CICIDS2017: F1 score per class

Class	Test windows	CNN-LSTM F1	Random forest F1
BENIGN	439,987	0.991	0.997
DoS Hulk	35,038	**0.990**	0.953

Class	Test windows	CNN-LSTM F1	Random forest F1
DDoS	25,595	0.990	0.998
PortScan	23,378	0.971	0.967
DoS GoldenEye	2,058	0.816	0.988
FTP-Patator	1,219	0.773	0.994
DoS slowloris	1,083	0.725	0.842
DoS Slowhttptest	1,046	0.886	0.900
SSH-Patator	739	0.656	0.945
Bot	381	0.056	0.031
Web Attack - Brute Force	288	0.125	0.688
Web Attack - XSS	131	0.006	0.336
Web Attack - Sql Injection	5	0.000	0.286
Heartbleed	3	0.800	1.000
Infiltration	0	-	-

- The CNN-LSTM is clearly ahead only on DoS Hulk, and close to the forest on DDoS and PortScan, the attacks that produce long runs of similar flows.
- **Bot is a data or feature limit:** both models fail on it.
- **The web attacks are a limit of the CNN-LSTM:** with the same features, the forest reaches an F1 of 0.69 on web brute force, so the signal exists in single flows and the windowed model loses it. Why has not been tested.
- Heartbleed (3 windows) and SQL injection (5) are too small to measure, and Infiltration has no test windows at all, because all its test rows fall within the first nine rows of a day and no complete window ends on them. These are the classes locked to human review for lack of evidence.

The confusion matrix (Fig. 6.6) shows where the errors go. Almost all errors of the weak classes are confusions with benign traffic: 91% of Bot windows, 75% of web brute-force windows and 95% of XSS windows are classified benign, and a quarter of slowloris and a fifth of SSH-Patator windows likewise. Confusions between attack classes are rare.

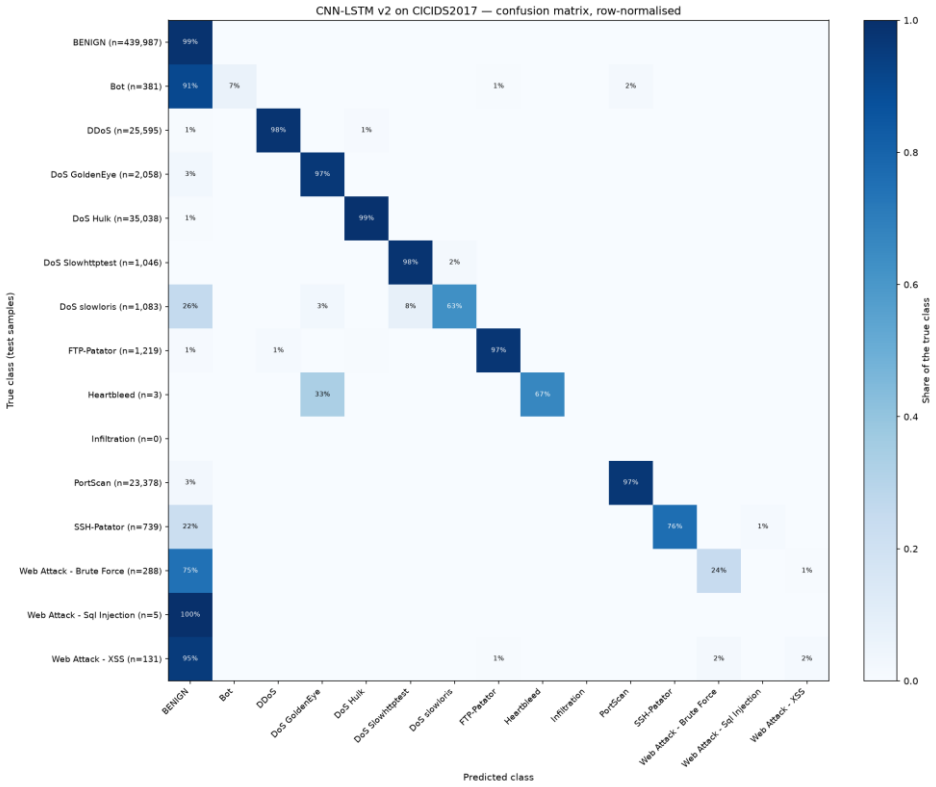


Fig. 6.6. Row-normalised confusion matrix of the CNN-LSTM on the CICIDS2017 test set

Fig. 6.7 shows the training curves of the deployed model. The validation loss was lowest after the first epoch and did not improve afterwards, so early stopping ended training after the sixth epoch and restored the first-epoch weights. Because the validation windows were a random 10% of the training windows, and neighbouring windows share nine of their ten flows, the validation set overlaps the training set and gives a weak stopping signal; the test score, which comes from the separate chronological test set, is not affected.



Fig. 6.7. Training and validation curves of the deployed CICIDS2017 model

6.3.2 Detection on MU-IoT

Table 6.4 compares the two models on the MU-IoT test windows whose exact sequence never appears in training (`model/results/mu_iot/`).

Table 6.4. MU-IoT: CNN-LSTM and random forest on unseen test windows

Measure	CNN-LSTM	Random forest
Within recordings: accuracy / macro F1 (528,967 windows)	**0.964 / 0.936**	0.904 / 0.816
Held-out recordings: accuracy / macro F1 (686,566 windows)	**0.680 / 0.676**	0.407 / 0.404
Normal traffic flagged as attack (within)	**9.3%**	17.2%
Held-out attacks classified as normal	**0.09%**	2.5%

On MU-IoT the CNN-LSTM is clearly better than the forest, most of all on the recordings it never saw. Within recordings its per-class F1 scores are higher for every class: DDoS 0.980 against 0.928, Injection 0.978 against 0.958, MiTM 0.963 against 0.862, Password_Hacking 0.990 against 0.955, Scan 0.994 against 0.949, Spyware 0.747 against 0.259 and normal 0.909 against 0.789. Table 6.5 shows the accuracy on each held-out recording.

Table 6.5. MU-IoT: accuracy on each held-out recording

Held-out recording	Class	CNN-LSTM	Random forest
MQTT dictionary attack	Password_Hacking	**0.998**	0.361
XSS attack	Injection	**0.882**	0.839
Vulnerability scan	Scan	0.446	**0.495**
hping3 UDP flood	DDoS	**0.321**	0.000

The CNN-LSTM almost never misses an attack on unseen recordings (0.09% called normal); its errors concern the name of the attack. Unfamiliar tools are mapped to the nearest familiar class: the hping3 flood, for example, is called Scan in 68% of its windows. Because every automatic MU-IoT action blocks the source address, a misnamed attack is still blocked. The main weakness is false alarms on normal traffic (9.3%), most of them predicted as Spyware.

Why sequences help on MU-IoT. With timestamps removed, identical MU-IoT flow records often appear with different labels: 71.8% of single Scan test records also appear in the training data with another label, against 0.0% of twenty-row Scan windows. A model that sees one record at a time cannot separate such

records, whatever its quality; a model that sees the sequence can. On CICIDS2017, by contrast, single flows are distinctive enough for a random forest to do better. The evidence therefore supports the CNN-LSTM where traffic is ambiguous at the level of single flows, as in the IoT data, not as a universal improvement.

6.3.3 Explainability

Choice of explainer for the dashboard. Table 6.6 summarises the measurements on which the dashboard's explainer was chosen.

Table 6.6. Measured comparison of SHAP explainers on the deployed model

Property	Gradient explainer	Kernel SHAP (deployed)
Additivity error (base value plus attributions against model output)	about 0.13	about 10^{-9} (largest measured 7.2 x 10^{-9})
Same input, repeated runs	Different results	Identical (fixed seed)
Time per window	1 to 16 s	about 1.5 s (1,000 samples)
Agreement with a converged reference	-	r = 0.98 with grouped features (r = 0.34 with 200 ungrouped inputs)

Two configuration details mattered. Grouping each feature's ten time steps into one value reduced the problem from 200 to 20 attributions, which the sampling budget can estimate reliably; and the explainer's default regularisation, which silently keeps only ten of the inputs, was switched off. The explanation is computed on demand for the selected window only, so its cost of about 1.5 seconds does not slow down the analysis.

Causes of the MU-IoT errors. Table 6.7 summarises the SHAP error analysis (`model/results/mu_iot/mu_iot_shap_report.txt`). In every group the mean SHAP sum matched the mean output to within 0.04.

Table 6.7. Causes of the main MU-IoT errors found with SHAP

Error	Similarity to the predicted class's detections	Similarity to the true class's detections	Cause
normal classified as Spyware	0.87	0.29	Systematic resemblance
hping3 flood classified as Scan	0.59	-0.64	Real resemblance to scans

Error	Similarity to the predicted class's detections	Similarity to the true class's detections	Cause
Vulnerability scan classified as Password_Hacking	0.20	-0.05	Extrapolation beyond the training range

- **normal classified as Spyware.** The same features (DNS_RCUL, TCPWS_Mode, FAMax, DNSQTL and FD) push both the false alarms and the real Spyware detections towards Spyware (Fig. 6.8), and the false alarms are confident, with a mean probability of 0.76. No confidence threshold can separate them from real detections, which is why Spyware is locked to human review.
- **hping3 flood classified as Scan.** The flood's packet sizes (about 64) and its near-zero FAMean feature match those of scans (packet size about 60), not those of the floods in the training data (packet size about 74, FAMean 0.017). The model is right that the traffic resembles a scan; the training data simply contain no flood of this kind.
- **Vulnerability scan classified as Password_Hacking.** One feature dominates: the mean TCP window size, with a median of about 25,000 in this recording against about 3,900 and 3,500 in the two classes. The model is extrapolating far beyond the values it was trained on.

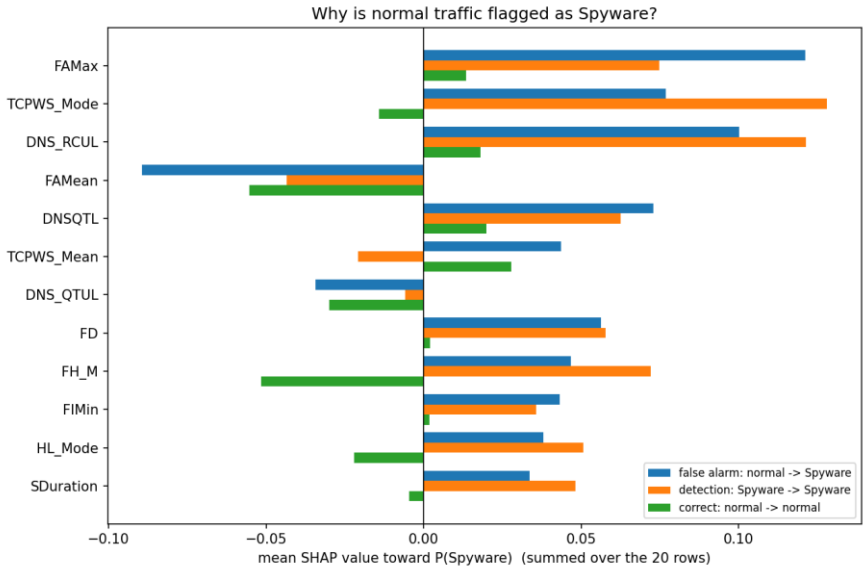


Fig. 6.8. Mean SHAP values of normal traffic flagged as Spyware, real Spyware detections and correctly classified normal traffic

6.3.4 Prevention Policy

Every test window was passed through the policy and the decision compared with its true label (`model/results/prevention_policy_cicids2017.txt` and its MU-IoT counterpart). Table 6.8 gives the result.

Table 6.8. Effect of the prevention policy on all test windows

Dataset	Benign windows actioned automatically	Attack windows handled automatically	Attack windows missed
CICIDS2017	0.33%	96.6%	2.4%
MU-IoT, within recordings	1.6%	94.1%	1.5%
MU-IoT, held-out recordings	no normal traffic	94.4%	0.08%

- **The measurement changed the policy.** Before Bot was locked to review, all 17 automatic Bot actions on the CICIDS2017 test set, at confidences of 0.94 and above, hit benign traffic. High confidence did not make these predictions right.
- On CICIDS2017 the policy handles 96.6% of attack windows automatically while acting on only one benign window in 300.
- On MU-IoT the false-action rate on normal traffic is five times higher (1.6%, mostly Injection), and it is the main remaining risk of the MU-IoT policy.
- The thresholds were derived from test-set F1 scores, and no separate calibration set exists, so these figures describe the policy rather than validate it independently.

6.3.5 Validation of the Flow Extractor

The original Friday capture of CICIDS2017, published by the University of New Brunswick, was passed through the project's extractor, and every resulting flow was compared with CICFlowMeter-V3's official row for the same flow (`model/results/extractor_validation.md`). The comparison covered ten minutes inside the PortScan period. Flows were matched one to one by their five-tuple and start time; the official timestamps turned out to be in local time (UTC-

3) and written without AM or PM, which the matching had to account for. Table 6.9 summarises the result.

Table 6.9. Extractor against CICFlowMeter-V3 on the original CICIDS2017 capture

Measure	Value
Flows matched one to one with official rows	5,957 of 7,332 (81%); the rest share a five-tuple within a minute, mostly reused nmap ports
All 20 features agree, at the precision printed in the official CSV	99.8%
All 20 features agree strictly (within 10^{-4})	86.3%
PortScan flows agreeing on all 20 features	49 of 49
Agreement if flows also end on RST	98.9% (so CICFlowMeter-V3 did not end flows on RST)

The official CSV prints large values with three significant digits (for example 1.02E+07), so strict agreement is lower than agreement at the printed precision; the per-feature agreement at the printed precision is between 99.83% and 100% for every one of the 20 features. The comparison found two real differences, both of which were corrected: times must be whole microseconds (floating-point epoch seconds had turned a 100 µs gap into 99.9 µs), and a FIN that starts a flow must not end it, so the reply joins that flow. The second correction is the TCP appendix artefact described by Engelen et al. [18]; correcting it raised strict agreement from 77.5% to 86.3%.

6.3.6 Real Traffic and Public Captures

Public captures. The data team ran the pipeline on public packet captures that the model was never trained on (Table 6.10, `docs/report/live_data_sources.md`). These captures have no window-level labels, so the predicted attack ratio shows the model's behaviour, not its accuracy. Three runs were cut at the dashboard's limit of 2,000 windows.

Table 6.10. Predicted attack ratio on public captures

Capture	Windows (processed)	Predicted attack ratio
CTU-13 Neris botnet, capture 42	2,369 (2,000)	23.2%
CTU-13 Neris botnet, capture 43	3,065 (2,000)	4.0%
SHIELD-i46 normal LAN traffic	105	about 1.0%

Capture	Windows (processed)	Predicted attack ratio
SHIELD-i46 5G DDoS traffic	2,849 (2,000)	94.2%
Small Wi-Fi capture (Wireshark)	7	14.3%

The DDoS capture is flagged almost entirely and the normal LAN capture almost not at all, which is the expected direction; the botnet captures, recorded in 2011 on a different network, are flagged far less consistently, which matches the model's weakness on the Bot class.

Own captures. Three captures of about three minutes each were recorded with a laptop as the protected host and an Android tablet as a second device on a phone hotspot (`model/results/hotspot_live_results.md`). The captures contain personal traffic and are not stored in the repository. Table 6.11 gives the results, together with the data team's ten-minute browsing capture.

Table 6.11. Results on own packet captures

Capture	Result
Port scan from the tablet (nmap -sT, 1,051 probes in 7 s)	98% of the windows pointing at the tablet classified PortScan; replayed through the live engine, the first detection came 1.6 s after the first SYN, and every automatic action targeted the tablet
Laptop browsing (IPv4 and IPv6)	11.1% of windows not benign; 7.6% recommend an automatic action, none with a target that could be blocked
Tablet repeatedly loading a page on the laptop	Windows classified Bot and held for review; no automatic action against the tablet
Ten-minute browsing capture (data team), live settings	29.9% of 4,857 windows not benign, 23.8% FTP-Patator

Four findings follow from these captures.

- **A scan is recognised only if the victim answers it.** In CICIDS2017 the victim servers replied to every probe. Windows' stealth mode replies to none, and the scan then looks benign to the model. For the demonstration, stealth mode was turned off on the laptop, so that it answers like the 2017 servers did.
- **Repeated polling of one web server reads as Bot,** because CICIDS2017's Ares botnet polls its command server in the same way. All 127 such

windows were held for review by the Bot lock: the policy, not the model, kept the tablet from being blocked.

- **Present-day browsing raises recommendation-only false alarms.** IPv6 and QUIC traffic, which did not exist in the 2017 training data in this form, is often classified as an attack (mostly DoS GoldenEye or FTP-Patator). The sources of these windows are the laptop itself and internet servers, never a local device, so nothing was executed. Analysing IPv4 only reduces the rates to 3.6% and 0%, but ignores 98% of the browsing packets. The rate also depends strongly on the traffic, from 11.1% to 29.9% between the two browsing captures.
- **The laptop sees its own traffic plus other devices' broadcasts.** Only a device that talks to the laptop can be a target (Section 4.5.7).

6.3.7 Live Run on a Real Network

On 18 October 2026 the whole system ran live on a home Wi-Fi router used daily by 20 to 30 people, with the owner's permission, and with the tablet as the only device listed as a permitted target. Table 6.12 summarises the run.

Table 6.12. Live run on a shared Wi-Fi network

Aspect	Result
Detection	PortScan at a confidence of 0.99, about 2 s after nmap's first probes; 13 consecutive PortScan windows
Prevention	Tablet blocked automatically with one rule; 18 further detections answered "already blocked"
Effect	The laptop's services (test page, dashboard, remote desktop) became unreachable from the tablet
Revocation	Page reachable again after revoking; three full scan, block and revoke cycles
Normal use	The tablet's page reloads read as Bot, held for review, never executed
Bystander	Another person's device, flagged as DoS Slowhttptest at 0.996 from its SSDP discovery broadcasts, refused 10 times because it was not listed

Most of nmap's 1,000 ports were probed within the second before the block took effect, so nmap still listed them as closed: a block stops an attacker from that moment on, not retroactively. The bystander is the most instructive result of the run. The model was wrong with high confidence, about a device that had nothing to do with the test, on a network used by other people; the allow-list refused

every attempt to act. The incident led to the rule that a device becomes a target only if it has sent traffic to the protected host, since discovery broadcasts never do.

The system was demonstrated again at the final project review on a different network, with the same sequence: live capture, port scan, automatic block, already-blocked reports, and revocation (Figs. 6.2, 6.4 and 6.5).

6.3.8 Inference Speed

Table 6.13 gives the inference speed measured on the laptop's CPU, plugged in, without a GPU (`model/results/inference_benchmark.txt`).

Table 6.13. Inference speed on the laptop CPU (AMD Ryzen 3 5300U)

Measure	CICIDS2017 model	MU-IoT model
One window, direct call (predict_on_batch)	1.48 ms	1.60 ms
One window, model.predict	67 ms	66 ms
Batches of 2,048 windows	about 70,700 windows/s	about 21,900 windows/s
Model file size	1.01 MB	-

The Keras `predict` method adds about 65 ms of fixed overhead per call. The extractor processes about 68,000 packets per second, and the live engine makes one prediction per second, so neither the model nor the extractor limits the live system; detection latency is set by the flow-ending rules and the one-second prediction interval.

6.4 Discussion

- **Sequences are not a universal improvement.** The comparison on identical examples is the central result: a report that evaluated the CNN-LSTM alone, or on CICIDS2017 alone, would have reached the wrong conclusion in one direction or the other. Sequences help where single flow records are ambiguous, as in MU-IoT.
- **Reliability, not accuracy, decides what may be automated.** An accuracy of 98.4% says nothing about whether a Bot prediction may be acted on; a Bot F1 of 0.056 does. The same review lock protected the tablet during the live run.

- **Safety limits are part of the result.** In the live run the model was confidently wrong about another person's device; only the allow-list prevented an automatic block.
- **Benchmark features must be reproduced exactly.** Validating the extractor against the original capture found two differences that no test based on the documentation would have found.
- **A 2017 model meets 2026 traffic.** None of the false alarms on present-day traffic was executed, but they show that the next step is retraining on current traffic, not a larger model.

6.5 Limitations

- **Model.** On CICIDS2017 a single-flow random forest outperforms the CNN-LSTM, which loses web-attack signal that single flows carry. Generalisation to unseen attack tools is weak (held-out macro F1 0.68). Normal traffic and Spyware in MU-IoT each come from one recording, so their scores are optimistic.
- **Evaluation.** Feature selection used a sample that includes some later test rows; the CICIDS2017 validation set overlaps the training windows; the CICIDS2017 test set contains later traffic of the same capture sessions, not unseen attacks; the policy thresholds come from test-set F1 without a separate calibration set; and each model was trained once, so the scores carry no error margin.
- **Extractor.** The extractor was validated on one ten-minute slice of one day, and only 49 PortScan flows could be matched one to one, because nmap reuses source ports.
- **Live operation.** One live attack type (a TCP connect scan) was run, on one network, from one device at a time. "Real time" means one prediction per second on the ten most recently ended flows; during a fast scan most windows are never classified, and blocking follows detection by about two seconds, by which time a fast scan has probed most ports.
- **Prevention.** Prevention acts only on documentation addresses and devices listed by hand; rate limiting is a temporary full block; input sanitisation against XSS is a recommendation only.

- **Dataset shift.** The 2017-trained model raises recommendation-only false alarms on present-day IPv6 and QUIC traffic (11% to 30% of browsing windows), reads repeated polling as Bot, and does not recognise scans against a silent, firewalled host.
- **Scope.** The MU-IoT model is evaluated offline and is not served by the dashboard, because live traffic is converted into CICIDS2017 features.

Conclusion and Future Enhancements

Conclusion

This project built a hybrid CNN-LSTM intrusion detection and prevention system and took it from benchmark data to live traffic on a real network. The model classifies windows of consecutive network flows into benign traffic and fourteen attack classes, the dashboard explains every prediction with SHAP, and a prevention layer blocks attacking devices through the host firewall.

Evaluated with leakage-controlled splits against a random forest on identical examples, the CNN-LSTM turned out to have specific strengths: the forest is better on CICIDS2017 (macro F1 0.780 against 0.586), while the CNN-LSTM is clearly better on the Indian MU-IoT dataset (0.676 against 0.404 on recordings never seen), because MU-IoT flow records are ambiguous on their own and sequences resolve them. SHAP separated the model's main errors on unseen IoT recordings into three different causes, one of which no confidence threshold can fix.

The prevention policy connects these measurements to action: thresholds follow each class's measured F1, unreliable classes are locked to human review, and on the CICIDS2017 test set the policy handles 96.6% of attack windows automatically while acting on 0.33% of benign windows. Execution is bounded by a kill switch, an allow-list, one rule per device and revocation that always works, verified by 35 automated tests, a mutation check (19 of 19 breakages caught) and an end-to-end check on the real firewall.

A packet-to-flow extractor reproduces the CICFlowMeter-V3 features on 99.8% of flows of the original CICIDS2017 capture, so live traffic and Wireshark captures are analysed with the training features. In a live run on a shared Wi-Fi network, a real port scan was detected about two seconds after its first probes and blocked automatically, while another user's device, flagged wrongly with high confidence, was refused by the safety checks.

The project's conclusion is therefore not that a deep model is always better, but that a deep model can be deployed responsibly when its evaluation is honest, its

errors are explained, and its authority to act is limited to what its measured reliability supports.

Future Enhancements

1. **Retraining on present-day traffic.** Fine-tuning the model on current benign traffic, including IPv6 and QUIC, would address the main source of false alarms found in the live tests.
2. **Further live attacks on an isolated laboratory network.** Floods, brute-force and web attacks need victim servers and several attacking machines; an isolated laboratory network would allow every class to be tested live, safely.
3. **A separate calibration set** for the prevention thresholds, so that the policy can be validated independently of the test set it was derived from.
4. **Repeated training runs** with different random seeds, to report the variance of every score.
5. **Recovering the web-attack signal.** The random forest detects web brute force from single flows (F1 0.69) where the CNN-LSTM does not; a model that combines the current flow's features with the window's sequence could keep both strengths.
6. **The MU-IoT model in the dashboard,** with its own extractor for the MU-IoT features, so that IoT traffic can be analysed live as well.
7. **True rate limiting** through Windows quality-of-service policies instead of a temporary full block.

References

[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2018, pp. 108-116, doi: 10.5220/0006639801080116.

[2] U. B. Clinton and N. Hoque, "MU-IoT: A new IoT intrusion dataset for network and application layer attacks analysis," *IEEE Access*, vol. 12, pp. 166068-166092, 2024, doi: 10.1109/ACCESS.2024.3494052.

[3] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," *Cybersecurity*, vol. 2, Art. no. 20, 2019, doi: 10.1186/s42400-019-0038-7.

[4] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 2, pp. 1153-1176, 2016, doi: 10.1109/COMST.2015.2494502.

[5] P. García-Teodoro, J. Díaz-Verdejo, G. Maciá-Fernández, and E. Vázquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," *Comput. Secur.*, vol. 28, nos. 1-2, pp. 18-28, 2009, doi: 10.1016/j.cose.2008.08.003.

[6] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2017, pp. 253-262, doi: 10.5220/0006105602530262.

[7] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5-32, 2001, doi: 10.1023/A:1010933404324.

[8] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278-2324, Nov. 1998, doi: 10.1109/5.726791.

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735-1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[10] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41525-41550, 2019, doi: 10.1109/ACCESS.2019.2895334.

[11] M. A. Ferrag, L. Maglaras, S. Moschoyiannis, and H. Janicke, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," *J. Inf. Secur. Appl.*, vol. 50, Art. no. 102419, 2020, doi: 10.1016/j.jisa.2019.102419.

[12] A. Halbouni, T. S. Gunawan, M. H. Habaebi, M. Halbouni, M. Kartiwi, and R. Ahmad, "CNN-LSTM: Hybrid deep neural network for network intrusion detection system," *IEEE Access*, vol. 10, pp. 99837-99849, 2022, doi: 10.1109/ACCESS.2022.3206425.

[13] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security Privacy*, 2010, pp. 305-316, doi: 10.1109/SP.2010.25.

[14] D. Arp *et al.*, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symp.*, 2022, pp. 3971-3988.

[15] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst. 30 (NeurIPS)*, 2017, pp. 4765-4774.

[16] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Security Defense Appl.*, 2009, pp. 1-6, doi: 10.1109/CISDA.2009.5356528.

[17] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, 2015, pp. 1-6, doi: 10.1109/MilCIS.2015.7348942.

[18] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE Security Privacy Workshops (SPW)*, 2021, pp. 7-12, doi: 10.1109/SPW53761.2021.00009.

[19] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Comput. Secur.*, vol. 86, pp. 147-167, 2019, doi: 10.1016/j.cose.2019.06.005.

[20] Y. Meidan *et al.*, "N-BaIoT: Network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Comput.*, vol. 17, no. 3, pp. 12-22, 2018, doi: 10.1109/MPRV.2018.03367731.

[21] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Gener. Comput. Syst.*, vol. 100, pp. 779-796, 2019, doi: 10.1016/j.future.2019.05.041.

[22] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165130-165150, 2020, doi: 10.1109/ACCESS.2020.3022862.

[23] A. A. Diro and N. Chilamkurti, "Distributed attack detection scheme using deep learning approach for Internet of Things," *Future Gener. Comput. Syst.*, vol. 82, pp. 761-768, 2018, doi: 10.1016/j.future.2017.08.043.

[24] N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, "A deep learning approach to network intrusion detection," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 2, no. 1, pp. 41-50, 2018, doi: 10.1109/TETCI.2017.2772792.

[25] C. Yin, Y. Zhu, J. Fei, and X. He, "A deep learning approach for intrusion detection using recurrent neural networks," *IEEE Access*, vol. 5, pp. 21954-21961, 2017, doi: 10.1109/ACCESS.2017.2762418.

[26] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 1135-1144, doi: 10.1145/2939672.2939778.

[27] O. Arreche, T. R. Guntur, J. W. Roberts, and M. Abdallah, "E-XAI: Evaluating black-box explainable AI frameworks for network intrusion detection," *IEEE Access*, vol. 12, pp. 23954-23988, 2024, doi: 10.1109/ACCESS.2024.3365140.

[28] D. Gaspar, P. Silva, and C. Silva, "Explainable AI for intrusion detection systems: LIME and SHAP applicability on multi-layer perceptron," *IEEE Access*, vol. 12, pp. 30164-30175, 2024, doi: 10.1109/ACCESS.2024.3368377.

[29] A. Nascita, G. Aceto, D. Ciuonzo, A. Montieri, V. Persico, and A. Pescapè, "A survey on explainable artificial intelligence for Internet traffic classification and prediction, and intrusion detection," *IEEE Commun. Surveys Tuts.*, vol. 27, no. 5, pp. 3165-3198, 2025, doi: 10.1109/COMST.2024.3504955.

[30] D. Manivannan, "A comprehensive analysis of explainable AI-driven machine learning techniques for network intrusion detection systems," *High-Confidence Comput.*, vol. 6, no. 4, Art. no. 100423, 2026, doi: 10.1016/j.hcc.2026.100423.

Appendix A

Backend API Reference

The Flask backend (`backend/app.py`) exposes the endpoints in Table A.1. All requests and responses use JSON, except the file uploads, which use multipart form data. No endpoint can enable firewall execution or change the set of permitted targets; both are fixed by environment variables when the backend is started.

Table A.1. Endpoints of the backend API

Method	Endpoint	Purpose
GET	/api/health	Model and SHAP status, SHAP background source, headline metrics, execution mode
GET	/api/meta	Feature columns in contract order, window length, dashboard categories, label mapping
POST	/api/predict	Prediction for one or more windows, with the prevention decision
POST	/api/explain	SHAP explanation of one window, with an additivity check
POST	/api/predict_csv	Batch analysis of an uploaded flow CSV
POST	/api/predict_pcap	Batch analysis of an uploaded Wireshark capture (.pcap or .pcapng, up to 200 MB)
POST	/api/explain_window	SHAP explanation of one window of the last upload
POST	/api/stream/load	Load a file for window-by-window replay
GET	/api/stream/next	Next window of the replay, with its prediction
GET	/api/live/interfaces	Network interfaces available for live capture
POST	/api/live/start	Start live capture on an interface
GET	/api/live/poll	New window results since the last poll, with counters
POST	/api/live/stop	Stop live capture
GET	/api/prevention/active	Firewall rules in force
POST	/api/prevention/revoke	Remove a rule created by the prevention engine

Appendix B

MU-IoT Features and Dashboard Categories

B.1 MU-IoT Features

Table B.1 lists the 38 MU-IoT features used by the MU-IoT model, in the order of the feature matrix. The names are those of the published dataset [2]; FPT and LPT, the times of a flow's first and last packet, were deliberately excluded (Section 3.2.3).

Table B.1. The 38 MU-IoT features

No.	Feature	No.	Feature	No.	Feature
1	FPC	14	ECN_M	27	MGA_UL
2	RTSP_ML	15	MQTT_MTL	28	FAMax
3	RTSP_MUL	16	DSCP_S	29	FH_M
4	ECE_FC	17	ECN_S	30	DSCP_UV
5	DNS_RCUL	18	TPackets	31	SDuration
6	FD	19	TCPWS_Mode	32	FlowR
7	DNS_QTUL	20	PLM	33	MGA_L
8	RTSP_MM	21	DSCP_M	34	PacketsPS
9	DNS_RCL	22	TCPWS_Mean	35	FIMin
10	DNSQTL	23	BJitter	36	ECN_C
11	URG_FC	24	BThroughput	37	FAMean
12	MQTT_MTUL	25	HL_Mode	38	RCount
13	HTTP_SCL	26	TCPWS_Sum		

B.2 Dashboard Categories

The dashboard groups the fifteen CICIDS2017 classes into nine operational categories that follow CERT-In's grouping of incidents (`backend/cert_in_mapping.py`), so that an analyst sees a short list of incident types instead of fifteen dataset labels (Table B.2). The categories are a display decision only; the model, the policy and all reported scores use the fifteen original classes.

Table B.2. Mapping of CICIDS2017 classes to dashboard categories

CICIDS2017 classes	Dashboard category
BENIGN	Normal
DDoS	DDoS
DoS GoldenEye, DoS Hulk, DoS Slowhttptest, DoS slowloris	DoS

CICIDS2017 classes	**Dashboard category**
PortScan	Port Scan
FTP-Patator, SSH-Patator, Web Attack - Brute Force	Brute Force
Web Attack - Sql Injection, Web Attack - XSS	Web Attack
Bot	Botnet
Heartbleed	Exploit
Infiltration	Infiltration